
Explorative Modeling: Unlocking a Third Pretraining Axis and End-to-End Generation

Alexi Gladstone¹, Heng Ji¹, Yilun Du²

¹UIUC ²Harvard

 explorative-modeling.github.io  github.com/alexiglad/XM

Abstract

The deep learning revolution, kicked off by AlexNet, taught us that end-to-end training beats decomposing a problem into hand-designed stages. Generative modeling, however, has remained the exception—despite generative models being remarkably capable, they are still not trained end-to-end.¹ This is because, at its core, *generative modeling is about handling multimodal distributions*,² and existing scalable approaches handle this multimodality the same way, by *factoring the generation procedure*, which prevents end-to-end generation. In this work, we introduce **Explorative Modeling**, a new paradigm that instead factors the training loop, exploring K candidate matches between model generations and data, and training on the best, so predictions commit to modes rather than blurring them. We find Explorative Models (XMs) useful in two settings. First, increasing exploration adds a **third pretraining axis beyond parameters and data** for existing generative models—where scaling exploration monotonically improves performance across both continuous and discrete domains (images, video, and language). Notably, gains from exploration **increase with scale**, climbing from 7% to 36% as data scales and from 13% to 23% as models grow, with efficiency gains more than doubling at $3\times$ the compute. Concretely, exploration improves FLOP efficiency by $4.1\times$, sample efficiency by $6.2\times$, parameter efficiency by 47%, lifts the strongest of image-generation recipes to a near-state-of-the-art 1.43 FID on ImageNet without guidance, enables *scaling how end-to-end existing models are*, and unlocks *scaling generalization*. Second, XMs enable end-to-end reconstructive generative modeling, matching diffusion on control tasks with $16\text{--}256\times$ fewer inference steps. Together, these results establish XMs as both a *new pretraining axis* for existing generative models and a standalone *end-to-end generative modeling paradigm*.

We scale the size of generative models and how much data we train them on... so why haven't we scaled what they can generate?

Correspondence to Alexi Gladstone:  alexigladstone@gmail.com. Work done while supported as a Flapping Airplanes Fellow.

¹The term end-to-end generative modeling is often used loosely. We provide a stricter definition in Section 2—simply put, sampling during training should be the same as sampling during inference.

²By *multimodal* we mean a probability distribution with many modes (distinct peaks), not data of different modalities such as text and images.

1 Introduction

AlexNet kicked off the deep learning revolution when *end-to-end* training beat hand-designed layer-wise training, demonstrating that *learning everything* performs better than hand-engineering [1]. Since then, end-to-end neural networks have broadly replaced many hand-built pipelines across the field including image classification [1, 2], object detection [3], and image segmentation [4], learning each task directly from data. Much of this success comes down to a single property: end-to-end models perform inference exactly as they were trained, so they are never exposed to inputs unlike those seen in training, which avoids distribution shifts and exposure bias that degrade performance and generalization [5–9].

Despite this trend, generative modeling has remained the holdout: its most common and scalable recipes today—*reconstructive* generative models (described in Section 2)—are not end-to-end, sampling completely differently at inference than during training. For example, autoregressive and diffusion models are trained to predict a single step, but used at inference as recurrent neural networks over hundreds to thousands of predicted tokens or denoising steps, so per-step errors feed into the next step, drifting inputs off the training distribution and compounding errors [8–12]. This shortcoming raises a simple question: “*Why can’t we train reconstructive generative models end-to-end?*”

We argue this is because generative modeling is fundamentally about handling *multimodal probability distributions*. To achieve this at scale, existing reconstructive models—whether autoregressive, diffusion [13], or single-step [14, 15]—factor the *generation procedure* into smaller steps during training, making each step’s target nearly unimodal so that a reconstruction loss no longer blurs distinct modes into their average. This factorization, however, is exactly what prevents current generative models from being end-to-end, so what else can we factor instead?

A generative model has only two processes to decompose—how it *generates* and how it *trains* (Figure 1). Because we’ve ruled out factoring generation, we factor the *training loop* itself—a new paradigm we call **Explorative Modeling**: at each training step, the model explores K possible matches between what it generates and the data, and trains on the closest. Because this happens entirely during training, Explorative Models (XMs) capture multimodal distributions while enabling end-to-end generation.

Exploration works by searching for which latent should be matched to which datapoint, something standard generation factorization completely sidesteps. In generative modeling, there is nothing that determines which datapoint each latent, such as input noise, should produce, so a latent is typically paired with targets at random. When models are trained to reconstruct many different valid targets at random, the best a single prediction can do is predict their average, a blur that matches no real datapoint (Figure 2 XM-1). XMs instead search for the latent whose generation is already closest to each datapoint, so each explored candidate can commit to a different mode, meaning the number of modes a model can capture, its *generative expressivity* (Section 2), grows directly with the amount of exploration.

As a consequence, we find XMs are valuable in two settings. First, added to existing generative models, exploration is a *new pretraining axis* beyond parameters and data. Existing generative models fix generative expressivity at training time through how they factor generation, so when they cannot capture every mode in the data, performance is capped no matter how far parameters and data scale. Because exploration scales generative expressivity directly, it relieves a bottleneck the other axes cannot, monotonically improving performance across both continuous and discrete domains—including images, video, and language. Crucially, like scaling parameters or data,³ these gains *grow with scale* rather than saturate, rising from 7% to 36% as data grows, 13% to 23% as models grow, and with efficiency gains more than doubling at $3\times$ the compute, so these numbers likely understate the gains at larger scale. Concretely, exploration improves FLOP efficiency by $4.1\times$, sample efficiency by $6.2\times$, parameter efficiency by 47%, lifts the strongest of image-generation recipes to a near-state-of-the-art 1.43 FID on ImageNet without guidance, enables a *compute-generalization tradeoff* where more exploration improves generalization, and increasing exploration enables existing generative models to be trained more end-to-end.

³Under compute-optimal scaling, parameters and data have to grow together: increasing one while holding the other fixed becomes increasingly suboptimal [16]. Our findings show that exploration largely acts the same way: as scale increases, models without exploration fall increasingly short of compute-optimal performance.

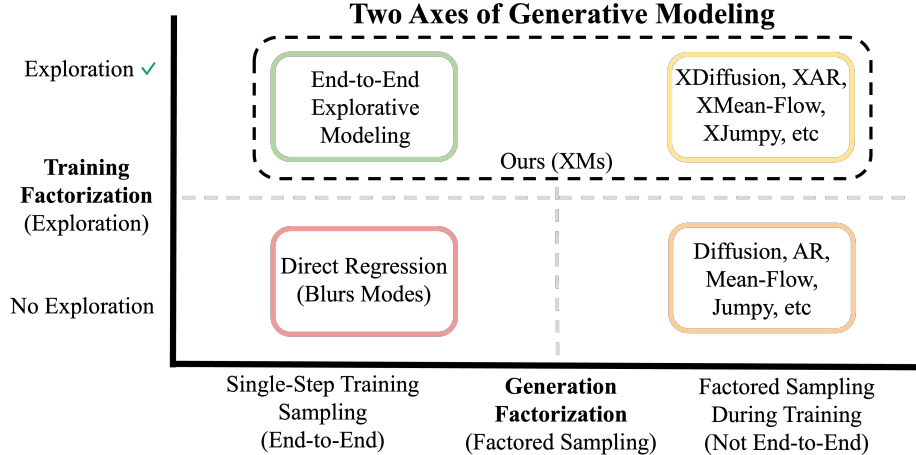


Figure 1: **Factorization Axes of Generative Modeling.** A generative model can factor either *generation* (x-axis) or *training* (y-axis). Factoring generation breaks sampling into many steps during training, making a model *not* end-to-end (right column); factoring training involves *exploration*, which trains on the modes a model captures best (top row). With neither, direct regression blurs distinct modes into their average (bottom left). Existing generative models factor generation but never training (bottom right), so adding exploration is a new pretraining axis for them (top right), while factoring training alone yields End-to-End Explorative Modeling (top left).

Second, as a standalone approach, exploration enables *end-to-end reconstructive generative modeling*. We find end-to-end XMs match Diffusion Policy [17] on behavior cloning and Diffuser [18] on goal-conditioned world modeling, while taking as little as a single forward pass in place of hundreds ($16\text{-}256\times$ fewer).

In summary, we make the following contributions:

- We introduce **Explorative Modeling**, a new paradigm for handling multimodal distributions that works by factoring the *training loop* instead of the generation procedure.
- We show exploration is a *new scaling axis* for existing generative models, with gains in FLOP, parameter, and sample efficiency that *increase with scale*.
- We find exploration enables a *compute-generalization tradeoff*, where spending more training compute on exploration directly improves generalization.
- We show factorizing training and generation are *substitutable*: as exploration increases, the optimal generative model becomes more end-to-end, indirectly improving generalization.
- We present a *scalable end-to-end reconstructive generative modeling approach*, matching diffusion on control tasks at $16 - 256\times$ less inference compute.

Ultimately, exploration lets us scale how end-to-end existing generative models are, and taken to its limit, makes generative modeling fully end-to-end—extending to generative modeling the end-to-end training that has driven the rest of deep learning.

2 Background

In standard supervised learning, such as classification or regression, each input generally has a single correct output, so a deterministic mapping is sufficient. Generative modeling has no such mapping: a request like “generate a dog” has no single right answer, as there are billions, or even infinitely many, valid dog images. These valid outputs are the *modes* of the data distribution, and this large number of modes is what makes generation hard, so capturing them is the central focus of generative modeling.

2.1 Mode Forcing

Generative models broadly fall into two families: *reconstructive* and *contrastive* [19]. Contrastive generative models, such as GANs [20] and contrastive-divergence energy-based models (CD EBMs) [21, 22], are trained by contrasting generated data against true data—leveraging relative

supervision from comparing samples with no explicit target—but have struggled with scalability. We therefore focus on *reconstructive* generative models—the most common family that has scaled best thus far—which are trained by mapping a self-produced input, such as noise or a corrupted sample, back to an explicit data target that supervises each prediction, and include autoregressive, diffusion [13], and flow [23] models. This pairing of an input with the target it should map to is called the *coupling*, and the challenge with reconstructive models is that we do not know this coupling beforehand, so a single input is typically coupled to many valid targets across the dataset. This one-to-many coupling is what causes mode blurring when doing generative modeling naively, as the reconstruction loss minimizer of many targets is the mean, which lands between modes and matches no real datapoints (demonstrated in Figure 2 for XM-1).

Recent work on **Mode Forcing** [19] points out that every scalable reconstructive model is built to dodge exactly this blur, with the central thesis that *modern generative modeling is the art of designing a reconstructive objective whose loss minimizer captures modes instead of averaging them*. Existing approaches for achieving this at scale function by *factoring generation* into a sequence of smaller, nearly unimodal steps, so no single prediction is forced to average across modes. For instance, autoregressive models reconstruct a target one element at a time, predicting each element from the ones already revealed, which means there is rich conditioning to make the prediction over the next element nearly unimodal. Diffusion and flow models [13, 23] instead reveal the target gradually through denoising: each step conditions on a slightly noisier version and predicts a slightly cleaner one, keeping every step nearly unimodal. This *factoring of the generation procedure* is why scalable generative modeling approaches succeed at generating high quality samples, whereas direct, single-step regression does not.

In general, scaling generative models has meant scaling just two axes: parameters or model size/FLOPs, which govern what a model can *represent*, and the amount of data and training length, which govern what a model can *learn*. Mode Forcing suggests these two axes miss a third capacity:

Generative Expressivity: The number of distinct modes a generative model’s training objective allows it to capture.

Unlike parameters and data, generative expressivity is set by the training objective itself, so it stays fixed no matter how far the other two axes scale. When an objective allows for capturing fewer modes than the data has, the surplus modes are not dropped but averaged, so a single prediction lands between them and matches no real datapoint (Figure 2 XM-1). Formally, letting $M(q)$ denote the mode count of a distribution q and $P_{\theta}(\cdot | c)$ the model’s sampling distribution given conditioning c , generative expressivity is the largest conditional mode count an approach’s loss minimizers can retain over any data distribution, $E \triangleq \sup_{p^*, c} \sup_{\theta^* \in \arg \min_{\theta} \mathcal{L}(\theta)} M(P_{\theta^*}(\cdot | c))$ (further details in Section F.1). Direct regressors demonstrate why this axis matters, as they have $E = 1$: even with **unlimited parameters and data**, their best possible output (loss minimizer) is still a single blurred mean of all the modes (Figure 2 XM-1). This is counterintuitive because direct squared-error regression is itself maximum likelihood under a fixed-variance Gaussian—the likelihood is maximized faithfully, just over a density with a generative expressivity of one, whose best fit to multimodal data is the mean. Therefore, this suggests that *the traditional notion of performing some form of maximum likelihood with sufficient data and parameters is not enough*, but rather that *generative expressivity* is an overlooked scaling axis. This may also explain why likelihood has long been observed to correlate poorly with sample quality [24], as likelihood measures how well a density is fit while generative expressivity determines how many modes that density can hold. As a consequence, the field’s primary goal of optimizing *likelihood alone* may be the wrong one to chase, and generative expressivity should be optimized alongside it. At its core, Explorative Modeling is a new way to increase generative expressivity (Figure 1)—exploring K candidates with a direct regressor raises generative expressivity to at least K , sharpening blurred means into distinct modes (Figure 2). Because factoring generation exists to supply this same quantity, factorizing generation and training are *substitutable*, which we confirm empirically in Section 4.

However, factoring generation does not remove this generative expressivity limitation entirely, as even highly scalable generative modeling approaches, such as diffusion and autoregression, can leave modes uncaptured when single predictions inside their factored procedure face many valid targets at once—that is, when the generative expressivity of those single predictions is too low. Notably, this phenomenon *worsens* with scale: as we add parameters and data, model expressivity and what

Increasing Exploration $K \rightarrow$ Increases Generative Expressivity $\rightarrow$

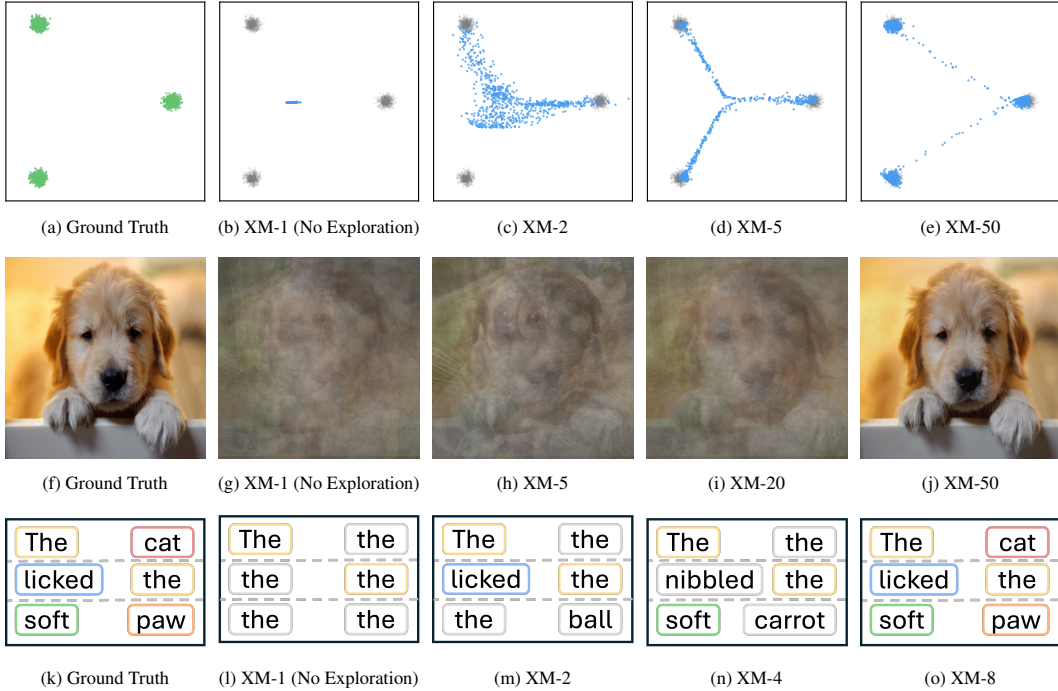


Figure 2: **Increasing Exploration Scales Generative Expressivity and Reduces Blurring.** Each row shows trained model generations varying only the amount of exploration (XM- K , denoting XMs with K modes explored): 2D mixture generation (top), image generation (middle), and masked diffusion language modeling (MDLM) [27] (bottom). With direct end-to-end regression (XM-1), models can only predict the mean of all samples—a single dot for three piles, a blurry image, and the word “the” repeated. XM-1 in the bottom row is the standard MDLM objective, which is prone to this collapse. As exploration increases, models become more *generatively expressive*, capturing the modes progressively better until generating high-quality samples in the right column.

models can learn cease to be the limiting factors, and generative expressivity increasingly becomes the bottleneck (we show this in Section 4). Evidence for this already exists in how heavily today’s models lean on guidance. Classifier-free guidance [25] sharpens samples by *pushing them away* from the unconditional model, which, with less conditioning to pin down each prediction, blurs modes more than the conditional model. This extrapolation helps primarily because the original model itself blurs modes. Autoguidance [26] reinforces this perspective even further, improving samples by pushing away from a deliberately worse, more mode-collapsed version of the model, demonstrating that the *core functionality of guidance is to push away from the conditional mean* due to challenges in capturing the true distribution. Together, this evidence points to the idea that even today’s best generative models may improve from added generative expressivity, which we confirm in Section 4.

2.2 End-to-End Generation

We call a generative model *end-to-end* when it samples the same way during training and inference, so it is never exposed to inputs at inference that it was not trained on.⁴ This is worth seeking for the same reason it has reshaped the rest of deep learning: ever since AlexNet [1], learning a task in an end-to-end manner has continued to beat splitting tasks into hand-designed stages. In generative modeling, this means learning the entire mapping from noise to data, including both its representations and its trajectory, rather than hand-specifying any part of it. End-to-end generative modeling also inherits the practical benefits of end-to-end training, where it removes *exposure bias* [8–12] and the train-inference mismatch that drifts a model onto out-of-distribution inputs and compounds errors [5–7], while as a byproduct enabling inference to be more efficient.

⁴This concerns the model’s own generation procedure, not test-time distribution shift—an end-to-end model can still face out-of-distribution test data, which is the ordinary generalization problem.

Yet general end-to-end *reconstructive* generation remains unsolved.⁵ Existing scalable reconstructive models capture modes by factoring generation into many steps, which makes training and inference inherently different—a diffusion model trained on a single denoising step is unrolled over hundreds of them at inference. Although recent methods push inference down to a single step [14, 15], during training they still anchor each prediction to the multi-step trajectory—conditioning on noised states or tying to the flow field—so that targets stay near-unimodal and avoid blurring. Their training therefore rarely simulates the one-step sampling they use at inference, so the train-inference mismatch remains and they are still not end-to-end.

2.3 Existing Generative Models

To study Explorative Modeling as a new scaling axis for existing generative models, our experiments build on two main families of generative models. First, we build on top of **Diffusion and Flow Matching** models [13, 23] which generate by repeatedly denoising via small steps from noise to data. Because Diffusion and Flow are equivalent formulations [28], we use the terms interchangeably throughout, and every Diffusion model over continuous data in this work is trained with the Flow Matching objective, as Flow has generally performed best [29]. Second, we experiment with **Jumpy** generative models [30], which generalize the idea of Diffusion/Flow by varying the number of steps, or *jumps*, interpolating between direct end-to-end regression (a single jump, the most end-to-end) and continuous-time flow (infinitely many jumps). This interpolation via the number of jumps enables a tradeoff between how end-to-end models are (fewer jumps) and how generatively expressive models are (more jumps). We return to this tradeoff when measuring how different models scale with exploration (Section 4).

3 Explorative Modeling Approach

3.1 Explorative Modeling Intuition

The goal of all reconstructive generative models is to design a training objective such that the loss minimizer captures modes instead of averaging them. The reason for this goal is demonstrated in Figure 2 for XM-1, where performing end-to-end direct regression with a naive training objective generates samples that do not belong to the data distribution. Existing generative models achieve this goal by factoring the *generation procedure* into a sequence of smaller steps, keeping each step’s target nearly unimodal so that no single prediction is forced to average across modes (Section 2.1). While these approaches have resulted in highly performant large-scale generative models [31, 32], they prevent models from being end-to-end (described in Section 2.2), which directly hurts performance and generalization due to exposure bias [8–12]. Therefore, instead of factorizing the generation procedure, the goal of Explorative Modeling is to enable a *new factorization axis*—the training loop itself (Figure 1).

In their simplest form, XMs are just *best-of- K* , an idea that has appeared many times in prior work [33–35] (Section E). At each training step, the model generates K candidate samples and trains on only the one closest to the data, implemented as a simple for loop (Algorithm 1) where only the best generation receives gradients. Formally, for a data sample x , generations $\hat{y}_1, \dots, \hat{y}_K \sim G_\theta$, and a reconstruction loss J such as squared error, the objective is

$$\mathcal{L}(\theta) = \min_{i \in \{1, \dots, K\}} J(\hat{y}_i, x). \quad (1)$$

Intuitively, the purpose of this for loop is to *change the loss minimizer from the mean of the data samples toward the true data samples themselves*. This matters because for most data, the mean of samples is not on the data manifold, as demonstrated in Figure 2. When the loss minimizer is the true data samples, models generate data that looks like real samples instead of a blurred, off-manifold average. Throughout this section we describe XMs as standalone models for intuition, though exploration can be added on top of existing generative models (Section 4.1).

There are several intuitions for what XMs are doing to enable handling multimodal distributions:

⁵Contrastive models such as GANs [20] and CD EBM [21, 22] have been end-to-end for a long time, but as noted in Section 2.1 they have struggled to scale.

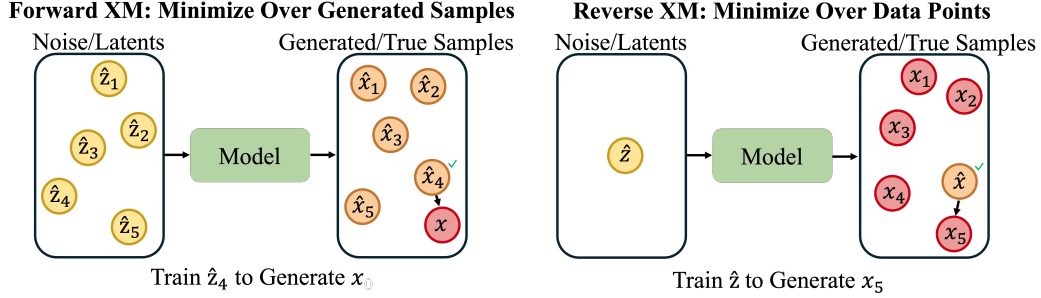


Figure 3: **Explorative Modeling Visualized.** Explorative Modeling explores possible matches between what the model generates and the data, and trains on the best match. This increases models’ *generative expressivity*, capturing multiple modes as opposed to predicting their mean (as in Figure 2 XM-1). In Forward XM, the model generates multiple samples that are compared to a ground truth sample. In Reverse XM, a generated sample is instead compared to many ground truth samples. In practice, both Forward and Reverse XM can be used together.

Scalable Training of Latent Variable Models Explorative Modeling can be seen as conditioning the generator on a *latent variable* (e.g., noise for diffusion models, or a learned embedding for language models) that resolves which of the many valid targets an input maps to: conditioned on the right latent, the one-to-many coupling (Section 2.1) becomes one-to-one, so the target becomes unimodal, and the blur disappears. The challenge with latent variable models is that we do not know which latent goes with which datapoint in advance. Variational Autoencoders (VAEs) [36] learn an encoder to infer this latent, which adds a KL term and risks posterior collapse, and consequently VAEs have struggled to scale well as standalone generative models [37, 38]. Explorative Modeling instead recovers the pairing through *exploration*, exploring possible matches between what it generates and the data and training on the closest. This trades extra training compute for end-to-end search of the latent variables.

A Scalable Way to Resolve Coupling Reconstructive models must pair each input with a target, referred to as a *coupling*, which we do not know in advance. Pairing at random is what ties one input to many targets, which blurs them (Section 2.1). Computing a better coupling directly does not scale: exact optimal transport is cubic in the number of samples, and its minibatch approximations [39] only match within a batch, a biased proxy for the true global pairing [40]. Instead of computing a global or minibatch coupling, Explorative Modeling searches for a coupling aligned with the model’s own samples. Because each pairing keeps only the best match rather than forcing an assignment over a whole batch, and searches the model’s own samples rather than fixed noise, the coupling avoids minibatch OT’s bias and co-adapts with the model throughout training.

Generative Modeling via Search Explorative Modeling can be seen as recasting generative modeling as a *search* problem, looking at training time for the latent, or coupling, that best explains the data. This framing is promising, as search and learning are the two methods the *bitter lesson* identifies as scaling well with computation [41].

Spreading Predictions Across Modes Geometrically, exploration changes what the best prediction strategy is. Consider guessing where darts land on a dartboard: with a single guess, the loss minimizer is the mean of all the throws, which is often a spot where few darts actually land. Forward XM (Equation 1) instead makes K guesses and scores only the closest, so the mean of the dartboard is no longer the loss minimizer—the best strategy becomes spreading the guesses so each covers a different cluster of throws. For the model, this means different latents specialize to different modes, so larger K captures more modes instead of blurring them together (Figure 2).

Minimizing an Implicit Energy Exploration can also be seen as implicit training-time energy minimization, where the loss acts as an *implicit energy* over pairings of a generation with the data, so searching for the lowest-loss match is a search for the minimum-energy, best-coupled samples. In this work, we search this landscape at random, but this search could instead be gradient-based (we describe this further in Section 6).

Algorithm 1: Forward XM (Minimize over Generated Samples)

Inputs: Generator G_θ , dataset $\mathcal{D}$, loss $J(\cdot)$

```

1 Sample  $x \sim \mathcal{D}$ ;
2 for  $i = 1, \dots, K$  do
3   | Sample  $\hat{y}_i \sim G_\theta$ ;
4   |  $\mathcal{L}_i \leftarrow J(\hat{y}_i, x)$ ;
5 return  $\min_i \mathcal{L}_i$ , update  $\theta$ ;
```

Algorithm 2: Reverse XM (Minimize over Data Points)

Inputs: Generator G_θ , dataset $\mathcal{D}$, loss $J(\cdot)$

```

1 Sample  $\hat{y} \sim G_\theta$ ;
2 for  $i = 1, \dots, K$  do
3   | Sample  $x_i \sim \mathcal{D}$ ;
4   |  $\mathcal{L}_i \leftarrow J(\hat{y}, x_i)$ ;
5 return  $\min_i \mathcal{L}_i$ , update  $\theta$ ;
```

3.2 Forward and Reverse Explorative Modeling

Exploration can search in either of two directions, which differ in what is held fixed and what is searched over (Figure 3).

Forward XM. Forward XM fixes a data target and explores its own generations: it draws K candidates and trains on the one closest to the target, exactly the best-of- K objective of Equation 1 (Algorithm 1), which we denote $\mathcal{L}_{\text{Forward}}$. Because every datapoint pulls in its nearest generation, no part of the data is ignored, so Forward XM is *mass-covering*—it errs toward *recall*, covering the full distribution. The challenge with Forward XM is compute—each of the K candidates is a separate generation, so covering more modes takes more forward passes.

Reverse XM. Reverse XM fixes a generated model sample and searches the data: it draws a single sample $\hat{y} \sim G_\theta$ and trains it toward the closest of K data targets $x_1, \dots, x_K \sim \mathcal{D}$ (Algorithm 2), flipping the objective to

$$\mathcal{L}_{\text{Reverse}}(\theta) = \min_{i \in \{1, \dots, K\}} J(\hat{y}, x_i). \quad (2)$$

Each generation is pulled onto the data manifold, so Reverse XM errs toward *precision*. Reverse XM is also cheap because it searches over data rather than generations, so each loss calculation only costs a single generation no matter how many targets it is compared against, which is useful for the large K values needed to handle highly multimodal data. Reverse XM’s weakness is that searching from the generation side applies no pressure to *cover every mode*, so on its own it is *mode-seeking* and can collapse onto a subset of the data. The two are therefore complementary—Forward XM focuses on recall/coverage whereas Reverse XM focuses on precision. In practice, the two can be combined to control for precision and recall. Moreover, exploration can also be added onto existing generative models and applied to partial, masked, or noised samples rather than only full generations, as in the hybrid XMs of Section 4.1. We discuss implementation details for both variants in Appendix C.

What Forward and Reverse XM Optimize. We can make the recall and precision behaviors of Forward and Reverse XM precise by asking what distribution each one drives the model toward. The starting point is that the squared error between a generation $\hat{y}$ and a datapoint x is, up to a constant, the negative log of a Gaussian $k_\sigma(\hat{y}, x)$ of width σ centered on the generation (σ is an analysis device set by the loss scale, not a hyperparameter). Each generation can therefore be seen as placing a small bump of density around itself, and averaging these bumps over everything the model generates gives the model a density of its own,

$$p_\theta(x) = \mathbb{E}_{\hat{y} \sim G_\theta} [k_\sigma(\hat{y}, x)] = (g_\theta * k_\sigma)(x),$$

where g_θ is the distribution of the model’s generations and $*$ is convolution, so p_θ is just g_θ blurred by the kernel. Blurring the data distribution p^* the same way gives $p_\sigma^* = p^* * k_\sigma$. In their smooth, large- K forms,⁶ Forward and Reverse XM then minimize

$$\underbrace{\text{KL}(p^* \| p_\theta) + H(p^*)}_{\text{Forward XM}} \quad \text{and} \quad \underbrace{\text{KL}(g_\theta \| p_\sigma^*) + H(g_\theta)}_{\text{Reverse XM}},$$

where KL measures the mismatch between two distributions and H is entropy. The two objectives are mirror images except for the entropy each carries. Forward’s entropy is the *data*’s, a constant the model cannot change, so **Forward XM is maximum likelihood** of the mixture its explored candidates form, and its large- K optimum recovers the data distribution up to that blur. Notably, this maximum likelihood reading holds at *every* K ; what changes with K is the density the likelihood is

⁶The smooth form scores the K candidates by $-\log \frac{1}{K} \sum_i k_\sigma(\hat{y}_i, x)$ rather than by the best alone, and differs from the hard min by at most $\log K$.

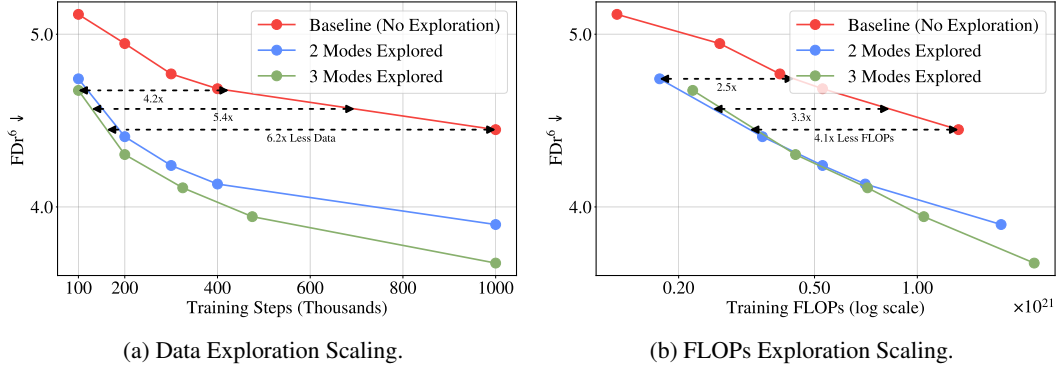


Figure 4: **Exploration Improves Data and FLOP Efficiency at Scale.** We add exploration to RAE [45], the state-of-the-art image generation recipe as of three months before this work (we report FDr⁶, as FID at this performance level is saturated [43, 44]). Exploration reaches the baseline’s best performance with 6.2× less data (Figure 4a) and 4.1× fewer FLOPs (Figure 4b)—more than doubling the gains of similar experiments using a third of the compute (Figure 5)—demonstrating that gains from exploration grow with scale.

fit over. At $K=1$ that density is a single Gaussian—the familiar fact that squared-error regression is Gaussian maximum likelihood—so the best the model can do is fit the blurred mean (Figure 2), while at larger K it is a mixture of the K candidates that can hold K modes, one per candidate. In other words, K scales how many modes the density can capture (its generative expressivity), which is why maximum likelihood alone can be misleading (Section 2.1). **Reverse XM targets the reverse KL**, the mode-seeking direction. However, the entropy in its objective is the *model’s* own, so the model can lower its loss simply by shrinking its spread, potentially resulting in collapse. One solution is to add an entropy bonus that cancels the model’s entropy term and leaves the pure reverse KL; another is to combine Reverse XM with Forward XM, which is mass-covering. We give precise statements, assumptions, and proof sketches for these claims in Appendix F.

4 Experimentation and Results

The goal of this section is to demonstrate that Explorative Modeling can be used both as a new pretraining axis (Section 4.1) as well as a standalone generative modeling approach (Section 4.2). As a new pretraining axis, we experiment with generative modeling hybrids combining exploration with either Diffusion/Flow [13, 23, 42] or Jumpy [30] generative models across both continuous and discrete domains. We refer to a generative model paired with Explorative Modeling by prefixing its name with an **X** (e.g., XDiffusion, XJumpy), reflecting how these are hybrid explorative and existing generative modeling combinations. Across all hybrid experiments, we do no XM-specific hyperparameter tuning, keeping each baseline recipe’s hyperparameters unchanged and only adding exploration. As a standalone generative modeling approach, we compare XMs to strong baselines in Behavior Cloning and Goal-Conditioned World Modeling. For all experiments in the main section of this paper, we denote exploring K modes as XM- K , and we use Forward XM (Section 3), as it is simpler to implement (this is discussed further in Section 6). Note that all baselines without exploration are equivalent to XM-1, as exploring a single mode reduces to standard training. By default, all experiments in this section are without guidance, except for guidance-based results reported in Table 1. Our largest image generation experiments report FDr⁶ because FID has saturated at this performance level [43, 44]; FDr⁶ works by averaging the Fréchet distance to the training data over six representation spaces.

4.1 Explorative Modeling as a New Scaling Axis

Does Exploration Improve Existing Generative Models’ Performance? Progress in generative modeling has largely been driven by scaling *parameter expressivity* through training larger models. This raises the question—if scaling parameter expressivity helps, *why not also scale generative expressivity*, a model’s capacity to capture multiple modes, rather than average them? Existing scalable reconstructive generative models rely on the same generation factorization regardless of model size, fixing generative expressivity at design time rather than scaling it. If this factorization is

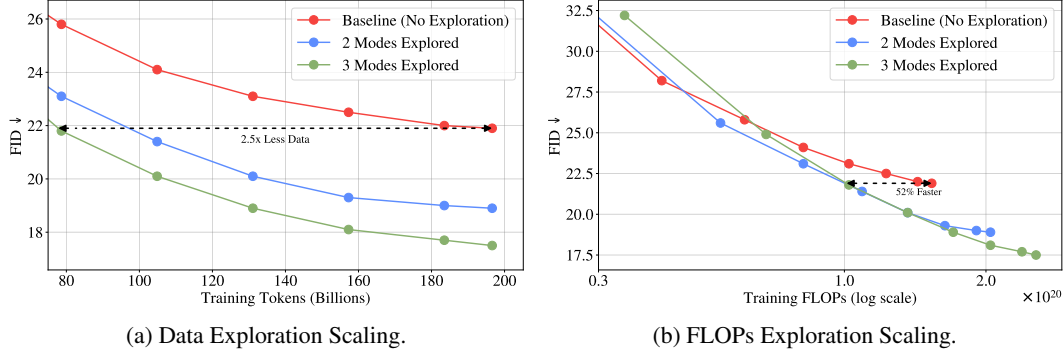


Figure 5: **Exploration Improves Sample and FLOP Efficiency.** We add exploration to an optimally tuned SiT baseline [29], training at roughly a third of the compute of Figure 4. Exploration reaches the same performance with $2.5\times$ less data (Figure 5a) and improves FLOP efficiency by as much as 52% (Figure 5b), with the compute-optimal amount of exploration increasing as models train longer—a trend that also holds at larger scale (Figure 4b).

not sufficient to capture all the modes in the distribution, we could expect that adding exploration, as a way to increase generative expressivity, could improve performance. To investigate this, we experiment with adding exploration to existing generative models, including Diffusion/Flow and Jumpy generative models [30] (which generalize Diffusion/Flow by varying the number of steps, or *jumps*, interpolating between single-step regression and continuous-time Flow; Section 2.3).

We begin by adding exploration to a strong image generation recipe (RAE [45]), training models that differ only in the amount of exploration. We find that exploration significantly improves performance throughout training, reaching the no-exploration baseline’s final performance with $6.2\times$ less data⁷ and $4.1\times$ fewer FLOPs (Figure 4). These gains also hold beyond a single recipe, where adding exploration to an optimally tuned SiT baseline [29] improves FLOP efficiency by as much as 52% and reaches the same performance with $2.5\times$ less data (Figure 5). Notably, the SiT experiments use roughly a third of the compute of the RAE experiments, meaning the efficiency gains from exploration *more than doubled* when moving to the larger-scale setting at $3\times$ the compute—suggesting gains from exploration grow with scale, a pattern we examine more directly below.

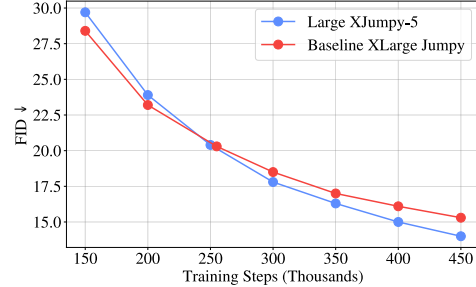


Figure 6: **Exploration Improves Parameter Efficiency.** A Large model with 5 modes explored *scales better* than an XLarge model with 47% more parameters and no exploration, demonstrating how exploration can improve parameter efficiency.

Notably, each explored mode in Forward XM adds compute (Reverse XM largely avoids this), yet despite this cost, the FLOP-optimal number of modes to explore grows as training continues (Figures 4b and 5b), as generative expressivity increasingly becomes the bottleneck. This mirrors compute-optimal parameter scaling [16], where just as the optimal number of parameters grows as compute increases, the optimal amount of exploration grows too, meaning *models that explore more modes eventually scale faster*. Exploration also improves parameter efficiency—a Large model exploring 5 modes outpaces an XLarge model with 47% more parameters and no exploration (Figure 6). These results show exploration is not just more performant, but that it enables a more efficient use of compute, data, and parameters.

Takeaway: Exploration improves existing generative models’ FLOP efficiency by $4.1\times$, sample efficiency by $6.2\times$, and parameter efficiency by 47%, with efficiency gains more than doubling when compute is tripled.

⁷Throughout this paper, we refer to two notions of sample/data efficiency. Here we mean the first, or the number of training samples processed (training steps at a fixed batch size) to reach a given performance. The second notion is the best performance achievable on a fixed-size dataset before overfitting, which concerns generalization; we test that separately in Figure 9.

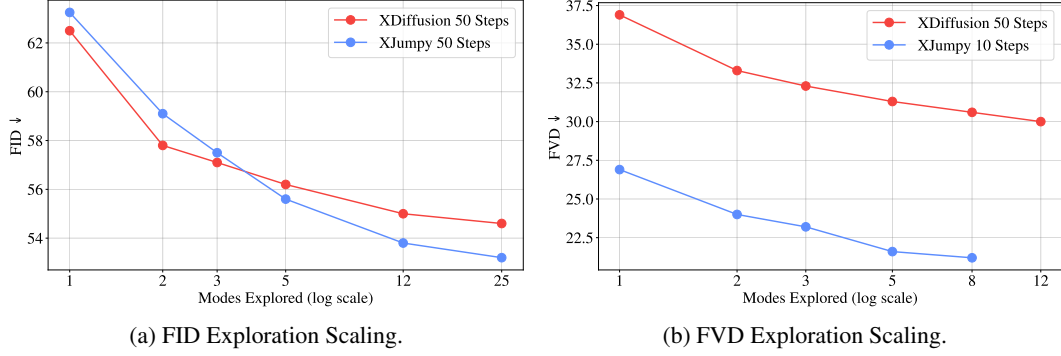


Figure 7: **Increasing Exploration Monotonically Improves Performance.** As the number of modes explored increases, both FID (left) and FVD (right) improve monotonically for Explorative Diffusion (XDiffusion) and Explorative Jumpy (XJumpy) models. In both cases, XJumpy benefits more from exploration than XDiffusion, a gap we examine in more detail below.

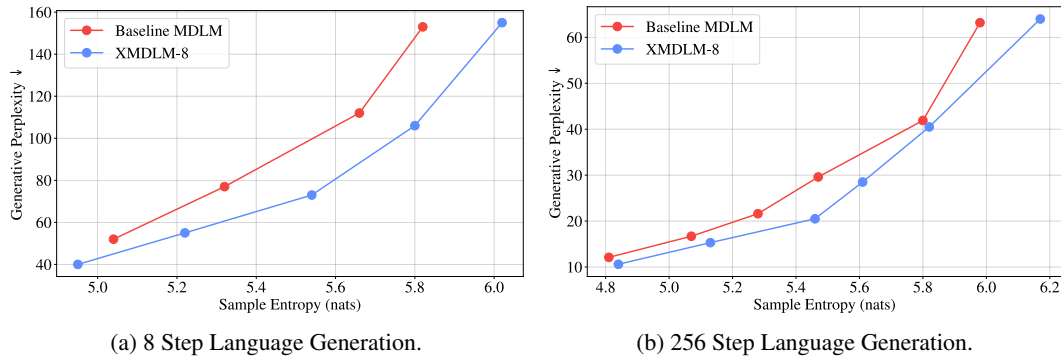


Figure 8: **Exploration Improves Masked Diffusion Language Modeling Performance.** Switching from a baseline Masked Diffusion Language Model (MDLM) [27, 48] to an Explorative MDLM (XMDLM) by exploring 8 modes significantly improves performance, achieving a better Perplexity-Entropy frontier for all points. This demonstrates exploration can improve generative models in both discrete and continuous spaces.

Does Performance Across Modalities Scale with Exploration? Having seen exploration added to existing models improve image generation performance, we next ask whether these benefits extend across modalities, and how they scale with the amount of exploration. To test this, we train image generation models, video generation models, and language models with a fixed parameter count, varying only the number of modes explored. For both image and video generation, increasing exploration monotonically improves performance as measured by FID and FVD respectively (Figure 7), with some models seeing a greater than 20% performance boost. This benefit carries over to discrete data, where adding exploration to a masked diffusion language model (MDLM) improves its perplexity-entropy frontier⁸ across the board (Figure 8), demonstrating that exploration helps in both continuous and discrete spaces. Notably, these gains *do not stop* as exploration increases (Figure 7), suggesting increased exploration could further improve performance.

Takeaway: Increasing exploration monotonically improves performance in both continuous and discrete spaces, for image, video, and language generation.

Does Exploration Improve Generalization? So far, we have measured sample/data efficiency as the number of training samples a model must process to reach a target performance. A stronger, more generalization-focused evaluation asks how much a model can extract from a *fixed* dataset—its best achievable performance before it begins to overfit [50]. We experiment with this setup, training models until their validation-set performance starts to get worse. One reason a model may overfit comes down to its generative expressivity—its capacity to represent multiple modes rather

⁸This frontier has become the standard evaluation in this setting, as generative perplexity alone can be gamed by low-entropy sampling [46, 47].

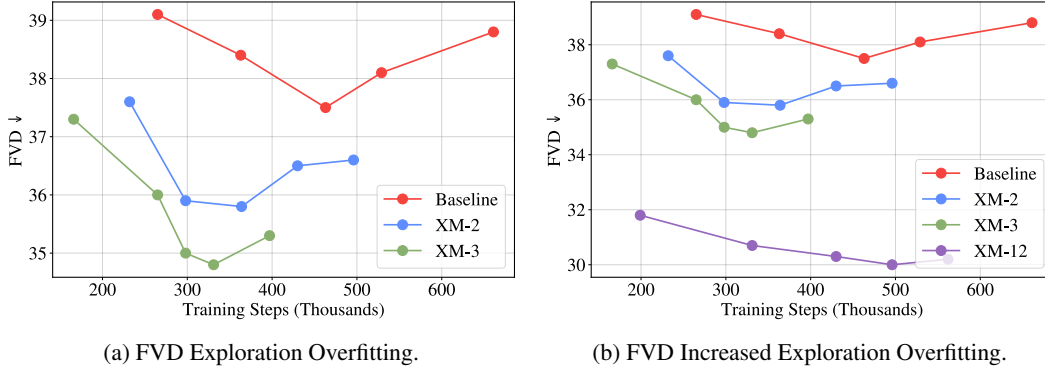


Figure 9: **Exploration Improves Generalization.** As the number of modes explored increases, 4-step XJumpy models achieve a better absolute minimum FVD due to overfitting less. The right panel shows the same runs as the left, adding the most-explored model (XM-12) to show this trend continues to the maximum exploration we test. Models overfit in this setting due to training on the relatively small Something-Something V2 dataset [49] (more on the setup in Section D).

than collapse them to an average (Section 2.1). With limited generative expressivity, a model’s best possible prediction is a blurred compromise between modes, which typically lies off the data manifold and matches no real datapoint, so a model fitting this compromise is memorizing something that does not exist in the true data distribution rather than generalizing. Even when generative expressivity is not the bottleneck, having surplus expressivity may ease optimization toward simpler solutions explaining the data, which tend to generalize better, much as overparametrization does [51, 52]. Therefore, as exploration increases generative expressivity directly, it may improve generalization. We find this for video generation (Figure 9), where increasing exploration improves generalization by reducing overfitting, resulting in a better absolute minimum FVD (30.0 with exploration versus 37.5 without exploration). Because this improvement comes from spending more training compute on exploration, it amounts to a *compute-generalization tradeoff*: extra compute directly buys better generalization. As data, rather than compute, increasingly becomes the bottleneck for large-scale training [50, 53, 54], we see improved generalization as an especially promising characteristic of XMs.

Takeaway: More exploration reduces overfitting, reaching better performance on a fixed dataset. This enables a *compute-generalization tradeoff*, where extra training compute directly buys generalization.

Does Exploration Improve State-of-the-Art Recipes at Scale? If exploration is a genuine scaling axis, it should improve even the strongest, most heavily tuned recipes. The Representation Autoencoder (RAE) [45] recipe from Figure 4 provides such a test: RAE was the state-of-the-art ImageNet 256×256 image generation recipe as of three months before the release of this work, primarily involving a change of representation space from the SD-VAE [32, 55] to a Representation Autoencoder. Aside from the previously discussed $6.2 \times$ data and $4.1 \times$ FLOP efficiency improvements, we find the performance gains also hold as models converge (training XL models for up to 2.2×10^{21} FLOPs), where an XM-2 RAE model reaches near-state-of-the-art non-CFG FID without post-training (Table 1), and much better FIDr^6 than the baseline. Convergence also compounds across recipes: XRAE converges $6.2 \times$ faster than RAE, which itself converges $47 \times$ faster than SiT [29, 45]—making XRAE almost $300 \times$ faster to converge than the standard SiT recipe.

Takeaway: Exploration improves even state-of-the-art recipes’ sample efficiency by more than $6 \times$, reaching a near-state-of-the-art unguided FID, and converging almost $300 \times$ faster than the standard SiT recipe.

Do Improvements From Exploration Vary With Scale? Throughout our experiments, exploration has helped more at larger scale—most notably, its efficiency gains more than doubled when moving from the SiT setting to the RAE setting, which used $3 \times$ the compute. This follows from how

Table 1: **Exploration Improves State-of-the-Art Image Generation Recipes.** We add exploration to the Representation Autoencoder (RAE) recipe [45] used for ImageNet 256x256. On the strongest RAE recipe, XRAE improves FDr⁶ in both the guided and non-guided settings and reaches a near-state-of-the-art non-guided gFID, demonstrating that exploration helps even the strongest, most heavily-tuned recipes at scale. We rely primarily on FDr⁶, which has started to become standard in this setting [43, 44], as gFID is highly saturated and often *misrepresents* sample quality [43]—so XRAE’s slightly worse guided gFID likely reflects this saturation. Table adapted from [45]; we report the RAE baseline performance under our setup.

Method	Generation@256 w/o guidance					Generation@256 w/ guidance				
	FDr ⁶ ↓	gFID↓	IS↑	Prec.↑	Rec.↑	FDr ⁶ ↓	gFID↓	IS↑	Prec.↑	Rec.↑
<i>Latent Diffusion with VAE</i>										
DiT [56]	-	9.62	121.5	0.67	0.67	-	2.27	278.2	0.83	0.57
MaskDiT [57]	-	5.69	177.9	0.74	0.60	-	2.28	276.6	0.80	0.61
SiT [29]	-	8.61	131.7	0.68	0.67	-	2.06	270.3	0.82	0.59
MDTv2 [58]	-	-	-	-	-	-	1.58	314.7	0.79	0.65
VA-VAE [59]	-	2.17	205.6	0.77	0.65	-	1.35	295.3	0.79	0.65
REPA [60]	-	5.78	158.3	0.70	0.68	-	1.29	306.3	0.79	0.64
DDT [61]	-	6.27	154.7	0.68	0.69	-	1.26	310.6	0.79	0.65
REPA-E [62]	-	1.70	217.3	0.77	0.66	-	1.15	304.0	0.79	0.66
<i>Latent Diffusion with RAE [45]</i>										
DiT ^{DH} -XL (DINOv2-B [63])	4.42	1.55	237.3	0.79	0.64	3.33	1.16	257.8	0.78	0.67
<i>RAE Recipe with Exploration (Ours)</i>										
XDiT ^{DH} -XL (DINOv2-B [63]), XM-2	3.91	1.43	240.3	0.79	0.64	3.17	1.19	254.9	0.77	0.67

a generative model’s performance is limited by three capacities: parameters restrict what it can represent, data restricts what it can learn, and generative expressivity restricts what it can generate. Conventional scaling of parameters and data relieves those constraints, but generative expressivity is set by the training objective itself (Section 2.1; see Section F.1 for the formal scope), so it stays fixed regardless of how large models or datasets grow. At small scale, this fixed generative expressivity is generally not an issue, as models are primarily held back by limited parameters and data. However, as parameter and data scale increase, generative expressivity increasingly becomes the bottleneck. Therefore, since exploration raises generative expressivity directly, we hypothesize its benefits should grow as models and data scale.

To test this, we measure the gains from exploration while varying model size and data scale, and find that gains rise from 7% to 36% as data scales, and from 13% to 23% as model size scales (Figure 10). The FLOP and sample efficiency experiments reinforce this—exploration helped more the longer a model was trained, with the FLOP-optimal amount of exploration growing over the course of training (Figures 4b and 5b). Together, these results point to exploration as a *missing scaling axis in existing generative models*, where the compute-optimal amount of exploration grows with scale just like parameters and data. This means today’s generative models, trained without exploration, *increasingly fall short* of what *compute-optimal exploration* would achieve. Foundation-model training runs use roughly four orders of magnitude more compute than our largest experiments, so if this trend continues, the improvements reported here likely underestimate the gains at that scale.

Takeaway: Gains from exploration grow with scale—as models and data scale, the bottleneck increasingly becomes generative expressivity, which exploration raises directly.

Which Models Benefit Most From Exploration? So far, exploration has helped every model family we tested, raising a natural question—are some generative models better suited to exploration than others?

To investigate this, we compare Diffusion/Flow with Jumpy generative models. Jumpy models work by interpolating between direct end-to-end regression (1 jump) and continuous-time Flow (∞ jumps), so using a finite number of jumps is more end-to-end than Flow and more *generatively expressive* than a single-step regressor (More details in Section 2.3).

We compare XDiffusion and XJumpy generative models for FID and FVD scaling as exploration increases in Figure 7, where the rate of improvement for XJumpy models as exploration increases is much higher than the rate for XDiffusion. For example, in Figure 7a, XJumpy generative models start

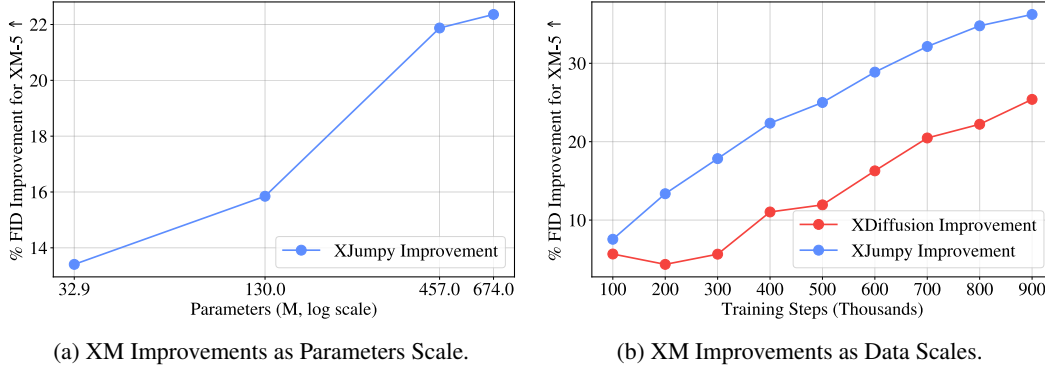


Figure 10: **Performance Gains from Exploration Increase as Scale Increases.** We measure the performance gain from exploring 5 modes (XM-5) over no exploration as we scale model size (left) and training data (right). In both cases the gains from doing exploration grow with scale, rising from 13% to 23% with model size and from 7% to 36% with data.

out as performing worse than XDiffusion models with no exploration, but as exploration increases XJumpy models become more performant than XDiffusion models. This is further reinforced by Figure 10b, where XJumpy models see larger gains from exploration than XDiffusion as data scales. Together, these results suggest that generative models that are more end-to-end scale better with increased exploration.

We can test this hypothesis further by varying the number of jumps within an XJumpy model. If factoring training can substitute for factoring generation, then more exploration should decrease the optimal number of jumps, since exploration supplies the generative expressivity those extra jumps would otherwise provide. Figure 11 shows this, where an XJumpy model with fewer jumps scales better with exploration than an XJumpy model with more jumps. These results directly demonstrate that as exploration increases, more end-to-end models perform better—in effect, *exploration scales how end-to-end existing generative models can be*. This makes Jumpy models, which can be more end-to-end than Diffusion/Flow, a promising approach to pair with exploration, and offers an indirect route to better generalization, as more end-to-end models reduce exposure bias (Section 2.2).

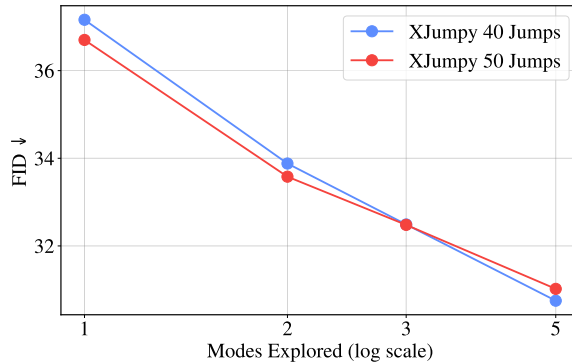


Figure 11: **More End-to-End Models Scale Better with Exploration.** We compare Explorative Jumpy (XJumpy) models with a different number of jumps across an increasing number of modes explored. If the amount of exploration is low, XJumpy models with more jumps perform best. However, as exploration increases, the optimal number of jumps decreases, demonstrating how models that are more end-to-end (fewer jumps) scale better with increased exploration.

Takeaway: Models that are *more end-to-end* benefit most from exploration, so as increased exploration supplies needed generative expressivity, the best-performing models become increasingly end-to-end—turning how end-to-end models are from a fixed design choice into a scalable one.

4.2 End-to-End Explorative Models

Can Explorative Modeling be Used for End-to-End Generation? So far, exploration has been combined with existing generative models, where we found it enables them to become more end-to-end (Figure 11); we now take this trend to its limit, using Explorative Models as *standalone* end-to-end generative models, where sampling is identical at training and inference (Section 2.2). We evaluate end-to-end XMs on robotics control tasks, including Behavior Cloning, comparing

Table 2: **Explorative Policy Rivals Diffusion Policy at $100\times$ Less Inference Compute.** Following the setup of Diffusion Policy [17], we report Behavior Cloning success rates on Robomimic tasks under the proficient-human, state-observation setting. Explorative Policy, our end-to-end Explorative Modeling-based policy, takes a single network forward pass (NFE: 1) at inference, while Diffusion Policy requires 100. Despite using significantly less inference compute, Explorative Policy matches or surpasses Diffusion Policy on all benchmarks.

Method	NFE↓	Lift↑	Can↑	Square↑	Transport↑	Tool Hang↑
<i>Proficient Human, State Observations</i>						
Diffusion Policy [17]	100	100%	100%	94%	72%	86%
Explorative Policy	1	100%	100%	96%	74%	86%

Table 3: **Explorative World Model Matches Diffuser While Using 16 – $256\times$ Less Inference Compute.** Goal-conditioned world modeling performance on the Maze2D tasks [18, 64]. Our Explorative World Model is compared against Diffuser [18], and achieves better average performance while using $80\times$ less inference compute on average. Some Explorative World Models take more than one NFE due to recurrent blocks [65].

Method	U-Maze		Medium		Large		Average	
	Score↑	NFE↓	Score↑	NFE↓	Score↑	NFE↓	Score↑	NFE↓
<i>Maze2D, Goal-Conditioned</i>								
Diffuser [18]	118.7	64	128.5	256	134.4	256	127.2	192
Explorative World Model	121.4	4	122.9	1	145.8	1.9	130.0	2.3

our Explorative Policy to Diffusion Policy [17] (Table 2), and Goal-Conditioned World Modeling, comparing our Explorative World Model to Diffuser [18] (Table 3).

In both settings, Explorative Models match diffusion baselines at a fraction of the inference compute—Explorative Policy rivals Diffusion Policy with a single network forward pass instead of 100, and the Explorative World Model matches Diffuser using 16-256 $\times$ fewer function evaluations. This gap comes directly from what each approach factors: diffusion pays for its generative expressivity with generation steps at inference, while end-to-end XMs pay for it with exploration during training, keeping inference at a single forward pass. Notably, we obtain these results with barely any tuning of hyperparameters for XMs—we keep each baseline’s architecture and occasionally add a recurrent block for an inductive bias toward recurrence—so we believe these results underrepresent how well-tuned XMs can perform with additional tricks. The main limitation of these experiments is in handling highly multimodal distributions. Because we use Forward XM here, which has a cost that grows with the number of modes explored, it cannot cheaply cover extremely multimodal distributions. Therefore, Reverse XM is likely better suited for end-to-end generation tasks, which we largely leave for future work (we discuss successful Reverse XM training in Section A, and future directions in Section 6).

Takeaway: Explorative Modeling enables scalable end-to-end reconstructive generative models, matching strong diffusion baselines on robotics and world modeling tasks while using up to $256\times$ less inference compute.

5 Discussion

Mode Forcing as a Predictive Theory. Much of deep learning progresses by running experiments first and explaining them afterward. Because this work builds on Mode Forcing [19], most of its results came about in the opposite order, where the theory predicted them before the experiments were run. Here we outline Mode Forcing’s predictions and their confirmations:

- **Even the strongest generative models are short on generative expressivity.** Mode Forcing argues that factoring generation often still leaves modes uncaptured, with heavy reliance on guidance as evidence for this (Section 2.1). Added generative expressivity should therefore improve even the most heavily tuned recipes, and it does: exploration lifts image, video, and language generation performance for all recipes tested.

- **Generative expressivity increasingly becomes the bottleneck at scale.** As parameters and data stop limiting what models can represent and learn, Mode Forcing predicts that generative expressivity set by the training objective should increasingly become the bottleneck. We find exactly this, where gains from exploration climb from 7% to 36% as data scales and 13% to 23% as models grow (Figure 10), and the FLOP-optimal amount of exploration rises over the course of training (Figures 4b and 5b).
- **Exploration can substitute for generation factorization.** As discussed in Section 2.1, factoring generation exists to supply generative expressivity, so supplying it through exploration instead should reduce how much generation factorization a model needs. We confirm this, where as exploration grows, the optimal amount of generation factorization decreases, and more end-to-end models perform better (Figure 11).
- **XMs enable end-to-end generation.** Mode Forcing argues that multimodal distributions are the core reason generation has to be factored, so if exploration handles multimodal distributions during training, end-to-end reconstruction should work. Our Explorative Policy and World Model confirm this, matching diffusion baselines with as little as a single forward pass instead of hundreds (Tables 2 and 3).

The Benefits of Surplus Generative Expressivity. Interestingly, exploration helps even when generative expressivity is not a large bottleneck. In our video generation experiments, XMs improve performance significantly even though the modeled distribution is not very multimodal—Jumpy models need only 10 steps in this setting (Figure 7), far less generation factorization than modern Diffusion models. Why would exploration help when there are few modes to capture? Our hypothesis is that even a weakly multimodal target still pulls each prediction toward multiple competing values over the course of training, and the same conflicting pulls that blur modes also make optimization harder. Exploration relieves this pressure, letting each prediction train toward its nearest match, so targets compromise less and optimization becomes easier. This mirrors overparametrization, where models with far more parameters than needed to fit their data consistently perform and generalize better [53, 54]—commonly attributed to smoother loss landscapes and a bias toward simpler solutions [52, 66]. In both cases, surplus capacity makes good solutions easier to find, suggesting that exploration, like parameters, is worth scaling past the point where it seems strictly necessary.

6 Future Works and Broader Impact

XMs open several research directions, below we highlight some of these directions.

Exploration as a Scaling Axis for More Generative Models. We demonstrated exploration acts as a scaling axis for Diffusion/Flow, Jumpy, and masked diffusion language models; we believe other generative models likely benefit from exploration in the same manner. Autoregressive LLMs have proven the hardest case, for the reasons discussed in our limitations (Section 7). Evaluation is also part of the challenge, as language modeling lacks robust distributional metrics like FID and FVD that would reveal mode coverage. We see two promising paths to build on these early gains. Multi-token prediction [67] targets are more multimodal, so multi-token prediction suffers more from limited generative expressivity and gives exploration more to offer. The Free Transformer [68] conditions a decoder on a latent variable inferred by a VAE, which is exactly the kind of latent exploration searches over, and training it with Explorative Modeling instead would remove the VAE entirely, along with the exposure bias of training on inferred latents (Section 3.1). Beyond language models, few-step models such as MeanFlow [15] are a natural fit for exploration as well, since exploration can supply the generative expressivity their shortened trajectories amortize. We also believe in combining XMs with Energy-Based Transformers (EBTs) [69], where the biggest documented challenge with EBTs has been end-to-end generation and handling highly multimodal distributions, which is exactly what XMs enable. Paired together, XMs and EBTs could enable more dynamic reasoning, search, and generalization over entire sequences.

End-to-end XM Applications. End-to-end XMs are especially well suited to new applications such as inpainting and super-resolution due to low amounts of multimodality in generated distributions. They could also pair with feature-based world models like JEPA [70] to build end-to-end world models. In the short term this pairing is especially practical for Forward XM, as feature spaces often

contain far fewer modes than raw observations [71], so a small K suffices (in the long run, we believe XMs can scale to arbitrarily multimodal settings via Reverse XM, Section A). Exploration would also resolve a core JEPA challenge: next-state prediction and trajectory-level planning are multimodal, especially in non-deterministic environments, which feature regression blurs but exploration captures. Another appealing direction is combining exploration with moment matching [72], which would let models reconstruct features at the right granularity. Finally, end-to-end XMs further enable setting the number of modes a model captures, which existing generative models struggle with, and because Forward XM favors recall while Reverse XM favors precision, choosing between or combining them gives direct control over generation diversity.

Improving and Understanding XMs. There is plenty of room to improve the core mechanism of exploration itself; in this paper we primarily used the simplest approach of sampling many different random noise candidates for Diffusion/Flow/Jumpy models. In principle, architectures could condition on discrete latent embeddings for each explorative factor, which could give better controllability, enable more uniform sampling of modes, and improve mode coverage (we did this for MDLMs, but no other models). Additionally, there are likely better exploration approaches that exist. In this work, exploration was done by drawing K independent candidates. In principle, however, searching for the optimal latent could be done in better ways, such as by treating the reconstruction loss as an energy, and finding the best latent by gradient descent [73]. The risk in using this approach is that it could cause a mismatch reminiscent of VAE prior holes [74], where the latents found by search differ from those sampled at inference, though applying such search only late in training or using other tricks could avoid this. Forward XM’s cost could also be cut with a cheaper scorer, such as a smaller proxy that ranks candidates so only the winner is generated in full. Training on the soft min rather than the hard min is another variant, letting every candidate contribute gradients and carrying the cleaner maximum likelihood interpretation (Section F.2). Another interesting idea would be to unify exploration with an end-to-end learned encoder, so search happens over learned latents (this could be combined with recent work on learning generative models and encoders jointly, such as Unified Latents [75]). Finally, exploration deserves the same scaling-law treatment as parameters and data: the compute-optimal amount of exploration already grows with scale (Figures 4b and 5b), so understanding how to optimally allocate compute between exploration, parameters, and data, similar to Chinchilla [16], would be insightful.

Scaling Reverse XMs. Despite Reverse XMs’ potential to collapse (Section 3.2), we see them as more promising in the long run over Forward XMs, as they add almost no extra FLOPs and scale more gracefully with the number of modes. With discrete conditioning, Reverse XMs come essentially for free, since loading a larger batch with K data points per condition lets each generation pick its best match (we could have done this for our image generation experiments, but chose not to in order to keep implementations simple and modality/domain agnostic). Doing Reverse XM with continuous conditioning is harder, as data points rarely share the exact same condition, so each generation has no ready-made set of valid targets to search, and the central design question becomes how data is loaded. More ambitiously, a vector database over the whole dataset would let each generation search all training data in logarithmic time, so the number of modes explored can in principle reach the dataset size, directly matching the generated distribution to the training distribution. We have already made Reverse XMs work this way on language modeling tasks (more on this in Section A). One remaining challenge is that a generation’s nearest datapoint can flip-flop across training steps, blurring the effective target; *sticky* couplings that persist matches across steps could prevent this.

Exploration beyond Pretraining. The mode collapse XMs address during pretraining also often shows up in post-training, where RL fine-tuning is known to sharpen models onto a narrow set of behaviors [76]. Recent fixes such as pass@ k rewards [77] and best-of- N -aware fine-tuning [78] can be seen through our lens as Forward XM, with a verifier standing in for ground truth data. These fixes act only during post-training, though; pretraining with exploration may yield base models that capture more modes in the first place, leaving RL more to select among.

7 Limitations and Conclusion

In this work, we introduced Explorative Modeling, a new paradigm for handling multimodal distributions that factors the training loop instead of the generation procedure. Exploration increases

generative expressivity, adding a new pretraining axis for existing generative models, and enabling end-to-end generative modeling.

Limitations. As a scaling axis for existing models, exploration is easier to integrate into some model families than others. This is because exploration requires a latent variable to search over when selecting the best of K candidates. Continuous generative models benefitted most easily, as they already condition on a noise z ; MDLMs benefitted once given a learned latent variable embedding for exploration. We found autoregressive language models harder to improve with exploration, likely because injecting a latent into them is less natural, and because they are less bottlenecked by generative expressivity than many other models. Despite this, we have achieved initial modest results showing improved data efficiency, suggesting exploration can benefit autoregressive LLMs further with more effort.

Exploration also changes the training objective, so losses are no longer directly comparable across exploration levels. This makes distributional metrics such as FID and FVD, as well as downstream metrics such as accuracy, more important for evaluation. Similarly, existing guidance techniques were designed without exploration in mind, and some transferred to XMs better than others: autoguidance worked decently, while classifier-free guidance helped less than it does for base models, despite XMs’ stronger unguided FID indicating they capture the underlying density better. Guidance is known to not transfer uniformly across models—for example, vanilla CFG also fails to improve models trained on representation autoencoder latents by default [45]—suggesting even our autoguidance results likely undershoot what XMs could achieve with guidance designed for exploration. Exploration also supplies a signal base models lack, namely K candidates and a notion of which was best, so we believe using it to design guidance tailored specifically to XMs is one of the most important open problems.

Fully end-to-end XMs face a couple of challenges. The most significant challenge with end-to-end XMs is in handling highly multimodal data: fully end-to-end Forward XMs need K to grow with the number of modes, which is currently too expensive for distributions with very many modes (e.g., image generation). Therefore, while the world remains somewhat compute constrained, Reverse XMs are a natural solution for handling high distribution multimodality, where a single model generation can search arbitrarily many training data points. Reverse XMs do bring their own considerations: being mode-seeking, they require an entropy term or coverage constraint to avoid collapse, and searching data efficiently requires good representations along with a dataloader or vector database supporting the search. Another challenge is that end-to-end XMs give up the implicit regularization of factored generation: when each step trains on corrupted or partial inputs, memorization is harder, so fully end-to-end models are more prone to memorizing when data is scarce. This concern fades with scale, however, as having abundant data itself acts as regularization [79–81]. Taken to the limit, with enough data and compute, it’s plausible that generative modeling simply becomes exploration for good latents using a very high K .

Conclusion. Across both continuous and discrete domains, we found exploration acts as a third pretraining axis alongside parameters and data, with gains that *grow with scale* rather than saturate—rising from 7% to 36% as data scales, 13% to 23% as models grow, and with efficiency gains more than doubling at $3\times$ the compute. Because gains from exploration keep climbing with scale, the numbers we report are likely a floor for benefits at increased model scale. Concretely, exploration improves FLOP efficiency by $4.1\times$, sample efficiency by $6.2\times$, and parameter efficiency by 47%, while lifting the strongest of image-generation recipes to a near-state-of-the-art 1.43 FID on ImageNet without guidance. Beyond efficiency, exploration enables *scaling generalization*: spending more training compute on exploration improves generalization directly, and improves it indirectly by enabling existing models to become more end-to-end. Taken to its limit, exploration enables fully end-to-end reconstructive generation, matching diffusion on control tasks with as little as a single forward pass in place of hundreds. For over a decade *we have scaled how large generative models are and how much data they train on*; XMs let us scale *what models can generate*.

Author Contributions

Alexi Gladstone led the project from ideation to execution, conceiving Explorative Modeling, developing the theory and method, designing and running all experiments, and writing the paper.

Heng Ji and Yilun Du advised the project throughout, providing invaluable mentorship, feedback on the ideas and writing, and compute support. Yilun had crucial initial ideas on XMs for training EBMs.

Acknowledgement

Huge thanks to Flapping Airplanes for supporting Alexi as a fellow while completing this work. Massive thanks to Laude Institute for supporting this work, with a special shoutout to Braden Hancock and K. Tighe. Thanks to Soran Ghaderi for productive early discussions related to XMs. Thanks to Omead Pooladzandi and Samip Dahal for great feedback on XMs. This material is based upon work supported by the U.S. National Science Foundation Graduate Research Fellowship Program under Grant No. DGE 21-46756, U.S. DARPA ECOLE Program No. #HR00112390060, DARPA ITM Program No. FA8650-23-C-7316, NSF Molecule Maker Lab Institute, an AI Institute for Molecular Discovery, Synthesis Strategy, and Manufacturing funded by the U.S. National Science Foundation under Awards No. 2019897 and 2505932, the AI Research Institutes program by National Science Foundation and the Institute of Education Sciences, U.S. Department of Education through Award No. 2229873 - AI Institute for Transforming Education for Children with Speech and Language Processing Challenges, and NSF NAIRR award. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the National Science Foundation, the Defense Advanced Research Projects Agency (DARPA), the Institute of Education Sciences, or the U.S. Department of Education. This research used the Delta and DeltaAI advanced computing and data resources, which are supported by the National Science Foundation (award OAC 2320345 and award OAC 2005572) and the State of Illinois. Delta and DeltaAI are joint efforts of the University of Illinois Urbana-Champaign and its National Center for Supercomputing Applications. Some of the computations in this paper were run on the FASRC cluster supported by the FAS Division of Science Research Computing Group at Harvard University.

References

- [1] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. *Advances in neural information processing systems*, 25, 2012. 2, 5
- [2] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016. 2
- [3] Nicolas Carion, Francisco Massa, Gabriel Synnaeve, Nicolas Usunier, Alexander Kirillov, and Sergey Zagoruyko. End-to-end object detection with transformers. In *European Conference on Computer Vision (ECCV)*, 2020. 2
- [4] Alexander Kirillov, Eric Mintun, Nikhila Ravi, Hanzi Mao, Chloe Rolland, Laura Gustafson, Tete Xiao, Spencer Whitehead, Alexander C Berg, Wan-Yen Lo, et al. Segment anything. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 4015–4026, 2023. 2
- [5] Benjamin Recht, Rebecca Roelofs, Ludwig Schmidt, and Vaishaal Shankar. Do imagenet classifiers generalize to imagenet? In *International Conference on Machine Learning (ICML)*, 2019. 2, 5
- [6] Dan Hendrycks and Thomas Dietterich. Benchmarking neural network robustness to common corruptions and perturbations. In *International Conference on Learning Representations (ICLR)*, 2019.
- [7] Pang Wei Koh, Shiori Sagawa, Henrik Marklund, Sang Michael Xie, Marvin Zhang, Akshay Balsubramani, Weihua Hu, Michihiro Yasunaga, Richard Lanus Phillips, Irena Gao, Tony Lee, Etienne David, Ian Stavness, Wei Guo, Berton A. Earnshaw, Imran S. Haque, Sara Beery, Jure Leskovec, Anshul Kundaje, Emma Pierson, Sergey Levine, Chelsea Finn, and Percy Liang. WILDS: A benchmark of in-the-wild distribution shifts. In *International Conference on Machine Learning (ICML)*, 2021. 5
- [8] Marc’Aurelio Ranzato, Sumit Chopra, Michael Auli, and Wojciech Zaremba. Sequence level training with recurrent neural networks. In *International Conference on Learning Representations (ICLR)*, 2016. 2, 5, 6
- [9] Samy Bengio, Oriol Vinyals, Navdeep Jaitly, and Noam Shazeer. Scheduled sampling for sequence prediction with recurrent neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2015. 2
- [10] Muru Zhang, Ofir Press, William Merrill, Alisa Liu, and Noah A. Smith. How language model hallucinations can snowball, 2023.
- [11] Mang Ning, Mingxiao Li, Jianlin Su, Albert Ali Salah, and Itir Onal Ertugrul. Elucidating the exposure bias in diffusion models. *arXiv preprint arXiv:2308.15321*, 2023.
- [12] Mingxiao Li, Tingyu Qu, Ruicong Yao, Wei Sun, and Marie-Francine Moens. Alleviating exposure bias in diffusion models through sampling with shifted time steps. *arXiv preprint arXiv:2305.15583*, 2023. 2, 5, 6
- [13] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020. 2, 4, 6, 9
- [14] Yang Song, Prafulla Dhariwal, Mark Chen, and Ilya Sutskever. Consistency models, 2023. 2, 6, 30
- [15] Zhengyang Geng, Mingyang Deng, Xingjian Bai, Zico Kolter, and Kaiming He. Mean flows for one-step generative modeling. *Advances in Neural Information Processing Systems*, 38:75460–75482, 2026. 2, 6, 16, 30
- [16] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 2022. 2, 10, 17, 35, 36
- [17] Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, page 02783649241273668, 2023. 3, 15, 29
- [18] Michael Janner, Yilun Du, Joshua B Tenenbaum, and Sergey Levine. Planning with diffusion for flexible behavior synthesis. *arXiv preprint arXiv:2205.09991*, 2022. 3, 15, 29
- [19] Alexi Gladstone, Yilun Du, and Heng Ji. Mode forcing: A unifying and predictive theory of generative modeling. Draft manuscript, 2026. URL https://alexiglad.github.io/assets/pdf/mode_forcing_draft.pdf. 3, 4, 15, 28, 31, 32

- [20] Ian J Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. *Advances in neural information processing systems*, 27, 2014. 3, 6, 30
- [21] Geoffrey E Hinton. Training products of experts by minimizing contrastive divergence. *Neural computation*, 14(8):1771–1800, 2002. 3, 6, 30
- [22] Yilun Du and Igor Mordatch. Implicit generation and modeling with energy based models. *Advances in neural information processing systems*, 32, 2019. 3, 6, 30
- [23] Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. *arXiv preprint arXiv:2210.02747*, 2022. 4, 6, 9
- [24] Lucas Theis, Aäron van den Oord, and Matthias Bethge. A note on the evaluation of generative models. In *International Conference on Learning Representations*, 2016. 4
- [25] Jonathan Ho and Tim Salimans. Classifier-free diffusion guidance. *arXiv preprint arXiv:2207.12598*, 2022. 5
- [26] Tero Karras, Miika Aittala, Tuomas Kynkäänniemi, Jaakko Lehtinen, Timo Aila, and Samuli Laine. Guiding a diffusion model with a bad version of itself. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. 5, 29
- [27] Subham Sekhar Sahoo, Marianne Arriola, Yair Schiff, Aaron Gokaslan, Edgar Marroquin, Justin T Chiu, Alexander Rush, and Volodymyr Kuleshov. Simple and effective masked diffusion language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2406.07524. 5, 11, 29
- [28] Ruiqi Gao, Emiel Hooeboom, Jonathan Heek, Valentin De Bortoli, Kevin P. Murphy, and Tim Salimans. Diffusion meets flow matching: Two sides of the same coin. 2024. URL <https://diffusionflow.github.io/>. 6
- [29] Nanye Ma, Mark Goldstein, Michael S. Albergo, Nicholas M. Boffi, Eric Vanden-Eijnden, and Saining Xie. Sit: Exploring flow and diffusion-based generative models with scalable interpolant transformers, 2024. 6, 10, 12, 13, 29
- [30] Alexi Gladstone, Yilun Du, and Heng Ji. Jumpy generative models: Unleashing the hidden spectrum in generative modeling. Manuscript in preparation, 2026. 6, 9, 10, 29
- [31] OpenAI. Gpt-4 technical report, 2023. 6
- [32] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 10684–10695, 2022. 6, 12
- [33] Stefan Lee, Senthil Purushwalkam, Michael Cogswell, Viresh Ranjan, David Crandall, and Dhruv Batra. Stochastic multiple choice learning for training diverse deep ensembles. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2016. 6, 30, 35
- [34] Ke Li and Jitendra Malik. Implicit maximum likelihood estimation. *arXiv preprint arXiv:1809.09087*, 2018. 31
- [35] Arash Vahabpour, Tianyi Wang, Qiuqing Lu, Omead Pooladzandi, and Vwani Roychowdhury. Diverse imitation learning via self-organizing generative models. *IEEE Transactions on Neural Networks and Learning Systems*, 36(4):7145–7157, 2024. 6, 30, 31, 35
- [36] Diederik P Kingma and Max Welling. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*, 2013. 7
- [37] Arash Vahdat and Jan Kautz. NVAE: A deep hierarchical variational autoencoder. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. 7
- [38] Zhisheng Xiao, Karsten Kreis, and Arash Vahdat. Tackling the generative learning trilemma with denoising diffusion gans. *arXiv preprint arXiv:2112.07804*, 2021. 7
- [39] Alexander Tong, Kilian Fatras, Nikolay Malkin, Guillaume Hugué, Yanlei Zhang, Jarrid Rector-Brooks, Guy Wolf, and Yoshua Bengio. Improving and generalizing flow-based generative models with minibatch optimal transport. *arXiv preprint arXiv:2302.00482*, 2023. 7, 26, 30

- [40] Kilian Fatras, Younes Zine, Rémi Flamary, Rémi Gribonval, and Nicolas Courty. Learning with minibatch Wasserstein: asymptotic and gradient properties. In *Proceedings of the Twenty Third International Conference on Artificial Intelligence and Statistics (AISTATS)*, volume 108 of *Proceedings of Machine Learning Research*, pages 2131–2141, 2020. 7, 26
- [41] Richard Sutton. The bitter lesson. *Incomplete Ideas (blog)*, 13(1):38, 2019. 7
- [42] Jascha Sohl-Dickstein, Eric Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics. In *International conference on machine learning*, pages 2256–2265. pmlr, 2015. 9
- [43] Jiawei Yang, Zhengyang Geng, Xuan Ju, Yonglong Tian, and Yue Wang. Representation Fréchet loss for visual generation. *arXiv preprint arXiv:2604.28190*, 2026. 9, 13, 26, 29
- [44] Jaskirat Singh, Boyang Zheng, Zongze Wu, Richard Zhang, Eli Shechtman, and Saining Xie. Improved baselines with representation autoencoders. *arXiv preprint arXiv:2605.18324*, 2026. 9, 13, 26, 29
- [45] Boyang Zheng, Nanye Ma, Shengbang Tong, and Saining Xie. Diffusion transformers with representation autoencoders. *arXiv preprint arXiv:2510.11690*, 2025. 9, 10, 12, 13, 18, 29
- [46] Kaiwen Zheng, Yongxin Chen, Hanzi Mao, Ming-Yu Liu, Jun Zhu, and Qinsheng Zhang. Masked diffusion models are secretly time-agnostic masked models and exploit inaccurate categorical sampling. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2409.02908. 11
- [47] Patrick Pynadath, Jiaxin Shi, and Ruqi Zhang. Generative frontiers: Why evaluation matters for diffusion language models. *arXiv preprint arXiv:2604.02718*, 2026. 11
- [48] Aaron Lou, Chenlin Meng, and Stefano Ermon. Discrete diffusion modeling by estimating the ratios of the data distribution. *arXiv preprint arXiv:2310.16834*, 2023. 11
- [49] Raghav Goyal, Samira Ebrahimi Kahou, Vincent Michalski, Joanna Materzynska, Susanne Westphal, Heuna Kim, Valentin Haenel, Ingo Fruend, Peter Yianilos, Moritz Mueller-Freitag, et al. The "something something" video database for learning and evaluating visual common sense. In *Proceedings of the IEEE international conference on computer vision*, pages 5842–5850, 2017. 12, 29
- [50] Mihir Prabhudesai, Mengning Wu, Amir Zadeh, Katerina Fragkiadaki, and Deepak Pathak. Diffusion beats autoregressive in data-constrained settings. *Advances in Neural Information Processing Systems*, 38:10581–10606, 2026. 11, 12
- [51] Preetum Nakkiran, Gal Kaplun, Yamini Bansal, Tristan Yang, Boaz Barak, and Ilya Sutskever. Deep double descent: Where bigger models and more data hurt. *Journal of Statistical Mechanics: Theory and Experiment*, 2021(12):124003, 2021. 12
- [52] Andrew Gordon Wilson. Deep learning is not so mysterious or different. *arXiv preprint arXiv:2503.02113*, 2025. 12, 16
- [53] Akshay Vegesna, Samip Dahal, Chinmay Karkar, Bishwas Mandal, Shmuel Berman, and Zhiwei Xu. Slowrun: Language modeling with infinite compute, fixed data, 2026. URL <https://github.com/qlabs-eng/slowrun>. 12, 16
- [54] Konwoo Kim, Suhas Kotha, Percy Liang, and Tatsunori Hashimoto. Pre-training under infinite compute. *arXiv preprint arXiv:2509.14786*, 2025. 12, 16
- [55] Stability AI. sd-vae-ft-mse, 2023. URL <https://huggingface.co/stabilityai/sd-vae-ft-mse>. Accessed: 2024-05-21. 12, 29
- [56] William Peebles and Saining Xie. Scalable diffusion models with transformers, 2023. 13, 28
- [57] Hongkai Zheng, Weili Nie, Arash Vahdat, and Anima Anandkumar. Fast training of diffusion models with masked transformers, 2023. 13
- [58] Shanghua Gao, Pan Zhou, Ming-Ming Cheng, and Shuicheng Yan. Mdtv2: Masked diffusion transformer is a strong image synthesizer, 2023. 13
- [59] Jingfeng Yao, Bin Yang, and Xinggang Wang. Reconstruction vs. generation: Taming optimization dilemma in latent diffusion models, 2025. 13
- [60] Sihyun Yu, Sangkyung Kwak, Huiwon Jang, Jongheon Jeong, Jonathan Huang, Jinwoo Shin, and Saining Xie. Representation alignment for generation: Training diffusion transformers is easier than you think, 2025. 13

- [61] Shuai Wang, Zhi Tian, Weilin Huang, and Limin Wang. Ddt: Decoupled diffusion transformer, 2025. 13
- [62] Xingjian Leng, Jaskirat Singh, Yunzhong Hou, Zhenchang Xing, Saining Xie, and Liang Zheng. Repa-e: Unlocking vae for end-to-end tuning with latent diffusion transformers, 2025. 13
- [63] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, Mahmoud Assran, Nicolas Ballas, Wojciech Galuba, Russell Howes, Po-Yao Huang, Shang-Wen Li, Ishan Misra, Michael Rabbat, Vasu Sharma, Gabriel Synnaeve, Hu Xu, Hervé Jegou, Julien Mairal, Patrick Labatut, Armand Joulin, and Piotr Bojanowski. Dinov2: Learning robust visual features without supervision, 2023. 13
- [64] Justin Fu, Aviral Kumar, Ofir Nachum, George Tucker, and Sergey Levine. D4rl: Datasets for deep data-driven reinforcement learning. *arXiv preprint arXiv:2004.07219*, 2020. 15
- [65] Jonas Geiping, Sean McLeish, Neel Jain, John Kirchenbauer, Siddharth Singh, Brian Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, and Tom Goldstein. Scaling up test-time compute with latent reasoning: A recurrent depth approach. *Advances in Neural Information Processing Systems*, 38:41340–41391, 2026. 15, 29
- [66] Hao Li, Zheng Xu, Gavin Taylor, Christoph Studer, and Tom Goldstein. Visualizing the loss landscape of neural nets. *Advances in neural information processing systems*, 31, 2018. 16
- [67] Fabian Gloeckle, Badr Youbi Idrissi, Baptiste Rozière, David Lopez-Paz, and Gabriel Synnaeve. Better & faster large language models via multi-token prediction, 2024. URL <https://arxiv.org/abs/2404.19737>. 16
- [68] François Fleuret. The free transformer, 2025. URL <https://arxiv.org/abs/2510.17558>. 16
- [69] Alexi Gladstone, Ganesh Nanduru, Md Mofijul Islam, Peixuan Han, Hyeonjeong Ha, Aman Chadha, Yilun Du, Heng Ji, Jundong Li, and Tariq Iqbal. Energy-based transformers are scalable learners and thinkers. *arXiv preprint arXiv:2507.02092*, 2025. 16, 30
- [70] Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture, 2023. 16
- [71] Yann LeCun. A path towards autonomous machine intelligence version 0.9. 2, 2022-06-27. *Open Review*, 62, 2022. 17
- [72] Yujia Li, Kevin Swersky, and Richard S. Zemel. Generative moment matching networks. In Francis R. Bach and David M. Blei, editors, *Proceedings of the 32nd International Conference on Machine Learning, ICML 2015, Lille, France, 6-11 July 2015, JMLR Workshop and Conference Proceedings*, pages 1718–1727. JMLR.org, 2015. URL <http://proceedings.mlr.press/v37/li15.html>. 17
- [73] Yilun Du, Shuang Li, Joshua Tenenbaum, and Igor Mordatch. Learning iterative reasoning through energy minimization. In *International Conference on Machine Learning*, pages 5570–5582. PMLR, 2022. 17
- [74] Mihaela Rosca, Balaji Lakshminarayanan, and Shakir Mohamed. Distribution matching in variational inference. *arXiv preprint arXiv:1802.06847*, 2018. 17
- [75] Jonathan Heek, Emiel Hoogeboom, Thomas Mensink, and Tim Salimans. Unified latents (ul): How to train your latents. *arXiv preprint arXiv:2602.17270*, 2026. 17
- [76] Anthony GX-Chen, Jatin Prakash, Jeff Guo, Rob Fergus, and Rajesh Ranganath. KL-regularized reinforcement learning is designed to mode collapse. *arXiv preprint arXiv:2510.20817*, 2025. 17
- [77] Zhipeng Chen, Xiaobo Qin, Youbin Wu, Yue Ling, Qinghao Ye, Wayne Xin Zhao, and Guang Shi. Pass@k training for adaptively balancing exploration and exploitation of large reasoning models. *arXiv preprint arXiv:2508.10751*, 2025. 17
- [78] Yinlam Chow, Guy Tennenholtz, Izzeddin Gur, Vincent Zhuang, Bo Dai, Sridhar Thiagarajan, Craig Boutilier, Rishabh Agarwal, Aviral Kumar, and Aleksandra Faust. Inference-aware fine-tuning for best-of-n sampling in large language models. *arXiv preprint arXiv:2412.15287*, 2024. 17
- [79] Gowthami Somepalli, Vasu Singla, Micah Goldblum, Jonas Geiping, and Tom Goldstein. Diffusion art or digital forgery? investigating data replication in diffusion models. *arXiv preprint arXiv:2212.03860*, 2022. 18

- [80] Zahra Kадkhodaie, Florentin Guth, Eero P Simoncelli, and Stéphane Mallat. Generalization in diffusion models arises from geometry-adaptive harmonic representations. *arXiv preprint arXiv:2310.02557*, 2024.
- [81] Xiangming Gu, Chao Du, Tianyu Pang, Chongxuan Li, Min Lin, and Ye Wang. On memorization in diffusion models. *arXiv preprint arXiv:2310.02664*, 2023. 18
- [82] Aram-Alexandre Pooladian, Heli Ben-Hamu, Carles Domingo-Enrich, Brandon Amos, Yaron Lipman, and Ricky TQ Chen. Multisample flow matching: Straightening flows with minibatch couplings. *arXiv preprint arXiv:2304.14772*, 2023. 26, 30
- [83] Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpthy, Aditya Khosla, Michael Bernstein, et al. Imagenet large scale visual recognition challenge. *International journal of computer vision*, 115:211–252, 2015. 27
- [84] Adam Casson. Transformer flops. 2023. URL <https://adamcasson.com/posts/transformer-flops>. 28
- [85] Uladzislau Sobal, Wancong Zhang, Kyunghyun Cho, Randall Balestriero, Tim GJ Rudner, and Yann LeCun. Learning from reward-free offline data: A case for planning with latent dynamics models. *Advances in Neural Information Processing Systems*, 38:43905–43941, 2026. 28
- [86] Tianhong Li and Kaiming He. Back to basics: Let denoising generative models denoise. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 36115–36125, 2026. 29
- [87] Tero Karras, Miika Aittala, Timo Aila, and Samuli Laine. Elucidating the design space of diffusion-based generative models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. 29
- [88] Mido Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, Mojtaba Komeili, Matthew Muckley, Ammar Rizvi, Claire Roberts, Koustuv Sinha, Artem Zhohus, Sergio Arnaud, Abha Gejji, Ada Martin, Francois Robert Hogan, Daniel Dugas, Piotr Bojanowski, Vasil Khalidov, Patrick Labatut, Francisco Massa, Marc Szafraniec, Kapil Krishnakumar, Yong Li, Xiaodong Ma, Sarath Chandar, Franziska Meier, Yann LeCun, Michael Rabbat, and Nicolas Ballas. V-jepa 2: Self-supervised video models enable understanding, prediction and planning, 2025. URL <https://arxiv.org/abs/2506.09985>. 29
- [89] Ajay Mandlekar, Danfei Xu, Josiah Wong, Soroush Nasiriany, Chen Wang, Rohun Kulkarni, Li Fei-Fei, Silvio Savarese, Yuke Zhu, and Roberto Martín-Martín. What matters in learning from offline human demonstrations for robot manipulation. In *Conference on Robot Learning (CoRL)*, 2021. 29
- [90] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, et al. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024. 29
- [91] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019. 29
- [92] Prafulla Dhariwal and Alexander Nichol. Diffusion models beat gans on image synthesis. *Advances in neural information processing systems*, 34:8780–8794, 2021. 30
- [93] Laurent Dinh, Jascha Sohl-Dickstein, and Samy Bengio. Density estimation using real nvp. In *International Conference on Learning Representations*, 2017. 30
- [94] Rob Cornish, Anthony L Caterini, George Deligiannidis, and Arnaud Doucet. Relaxing bijectivity constraints with continuously indexed normalising flows. In *International Conference on Machine Learning*, pages 2133–2143. PMLR, 2020. 30
- [95] Shuangfei Zhai, Ruixiang Zhang, Preetum Nakkiran, David Berthelot, Jiatao Gu, Huangjie Zheng, Tianrong Chen, Miguel Angel Bautista, Navdeep Jaitly, and Josh Susskind. Normalizing flows are capable generative models. In *International Conference on Machine Learning*, 2025. 30
- [96] Jiatao Gu, Ying Shen, Tianrong Chen, Laurent Dinh, Yuyang Wang, Miguel Angel Bautista, David Berthelot, Josh Susskind, and Shuangfei Zhai. Starflow-v: End-to-end video generative modeling with normalizing flows. *arXiv preprint arXiv:2511.20462*, 2025. 30
- [97] Kilian Fatras, Younes Zine, Szymon Majewski, Rémi Flamary, Rémi Gribonval, and Nicolas Courty. Minibatch optimal transport distances; analysis and applications. *arXiv preprint arXiv:2101.01792*, 2021. 30
- [98] Aram Davtyan, Leello Dadi, Volkan Cevher, and Paolo Favaro. Faster inference of flow-based generative models via improved data-noise coupling. In *International Conference on Learning Representations*, volume 2025, pages 60922–60947, 2025. 30

- [99] Yexiong Lin, Yu Yao, Yang Zhou, and Tongliang Liu. Beyond optimal transport: Model-aligned coupling for flow matching. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 3955–3964, 2026. 30
- [100] Abner Guzmán-Rivera, Dhruv Batra, and Pushmeet Kohli. Multiple choice learning: Learning to produce multiple structured outputs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2012. 30
- [101] Christian Rupprecht, Iro Laina, Robert DiPietro, Maximilian Baust, Federico Tombari, Nassir Navab, and Gregory D Hager. Learning in an uncertain world: Representing ambiguity through multiple hypotheses. In *International Conference on Computer Vision (ICCV)*, 2017. 30
- [102] Nisan Stiennon, Long Ouyang, Jeffrey Wu, Daniel Ziegler, Ryan Lowe, Chelsea Voss, Alec Radford, Dario Amodei, and Paul F Christiano. Learning to summarize from human feedback. *Advances in neural information processing systems*, 33:3008–3021, 2020. 30
- [103] Nanye Ma, Shangyuan Tong, Haolin Jia, Hexiang Hu, Yu-Chuan Su, Mingda Zhang, Xuan Yang, Yandong Li, Tommi Jaakkola, Xuhui Jia, et al. Inference-time scaling for diffusion models beyond scaling denoising steps. *arXiv preprint arXiv:2501.09732*, 2025. 30
- [104] Yuri Burda, Roger Grosse, and Ruslan Salakhutdinov. Importance weighted autoencoders. *arXiv preprint arXiv:1509.00519*, 2015. 33, 34
- [105] Richard S Sutton, Andrew G Barto, et al. *Reinforcement learning: An introduction*, volume 1. MIT press Cambridge, 1998. 36

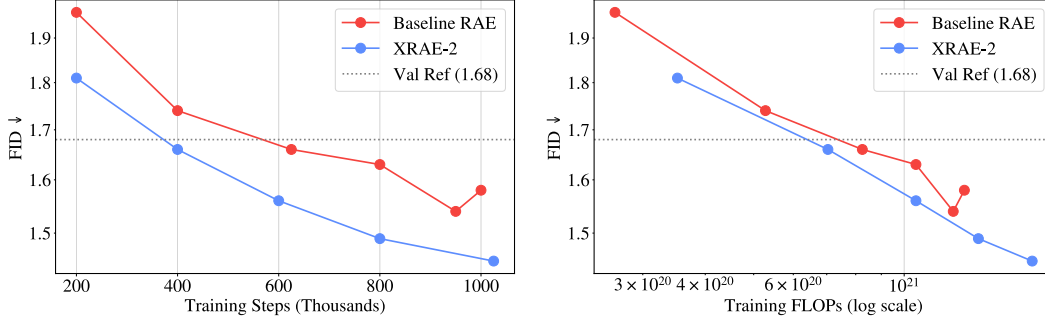


Figure A.1: **RAE FID Convergence.** The same comparison as Figure 4 measured with FID rather than FDr^6 : XRAE converges much faster in terms of FLOPs and training steps than the baseline RAE. The improvement gap is higher for FDr^6 , as FID at this level of performance is saturated and no longer tracks true sample quality [43, 44].

A Additional Experimentation

Figure A.1 reports the FID versions of the FDr^6 convergence plots in Figure 4, where XRAE similarly converges much faster than the baseline.

Though we do not report them in detail, we have also observed these benefits in the discrete domain, where adding exploration to MDLMs improves both sample efficiency and generalization, mirroring the trends for image and video generation (Figures 4 and 9). Similarly, we have observed early evidence that exploration improves the data efficiency of autoregressive language models, which we leave to future work to report in detail.

A.1 Exact Results for Exploration Scaling

For reproducibility, and to make future comparisons easier, Table A.1 reports the exact FID and FVD values behind Figure 7.

Table A.1: **Exact FID and FVD Values Across Exploration Scaling.** The values underlying Figure 7, reported to ease reproduction and comparison in future work. Dashes mark exploration levels not tested for that setting.

Model	Modes Explored (XM-K)						
	1	2	3	5	8	12	25
<i>FID↓, ImageNet 256×256, Small models (Figure 7a)</i>							
XDiffusion 50 Steps	62.5	57.8	57.1	56.2	–	55.0	54.6
XJumpy 50 Steps	63.3	59.1	57.5	55.6	–	53.8	53.2
<i>FVD↓, Something-Something V2, Base models (Figure 7b)</i>							
XDiffusion 50 Steps	36.9	33.3	32.3	31.3	30.6	30.0	–
XJumpy 10 Steps	26.9	24.0	23.2	21.6	21.2	–	–

A.2 Comparing Exploration to Minibatch Optimal Transport Couplings

A natural alternative to exploration is to reduce mode blurring by computing a better coupling directly, most commonly with minibatch Optimal Transport (OT) [39, 82]. Comparing the two, we find minibatch OT couplings actually *hurt* image generation performance, worsening FID from 46.3 to 54.5 for Small models and from 74.4 to 82.7 for Base models (Table A.2).

We attribute this to two problems that exploration avoids. First, minibatch OT is a biased approximation of the global coupling [40], and this bias worsens as datasets grow, since each batch covers a vanishing fraction of the data. Second, the two select couplings on different grounds. Minibatch OT assigns pairs by geometry alone, computed within a batch regardless of what the model has

learned; exploration selects by the model’s own current loss, so its coupling co-adapts with the model throughout training. Empirically, the model-aligned choice helps while the geometric one hurts, and exploration’s gains *grow* with scale rather than degrade (Figure 10).

Table A.2: **Minibatch OT Couplings Hurt Performance.** FID after 200k training steps for Flow Matching models trained with and without minibatch OT couplings. OT hurts performance at both model sizes, which we attribute to the bias of minibatch couplings and their model-agnostic assignment. The setup here is ImageNet-1k [83] class-conditional image generation.

Method	Small FID↓	Base FID↓
Flow Matching	46.3	74.4
Flow Matching + Minibatch OT	54.5	82.7

A.3 Reverse XM Language Models

We have successfully trained Reverse XM language models, where each generation searches the training data through a vector database rather than comparing against K samples in a batch. Getting this to work relied on two ingredients. First, once a data point is matched and trained on, it is removed from the search pool for that epoch, which we track as a *train coverage percent*. This prevents nearest-neighbor search from collapsing onto the same few points, playing the role of the entropy term Reverse XM needs to avoid collapse (Section F.2), though as a coverage heuristic rather than a literal entropy bonus. Additionally, we found it important to set the *train coverage percent* to be less than the entire train set (e.g., 50%), or else models have high loss near the end of an epoch as they try to cover the extremes of a dataset. Second, the search happens in a hybrid of representation space and cross-entropy space, so that retrieved neighbors reflect the actual training loss. We strongly believe this can be further improved upon, and we have not yet open sourced this code, but plan to.

B Additional Intuition

Generative Modeling with a For Loop. It is worth appreciating how simple end-to-end Explorative Modeling can be. Unlike diffusion, flow, or autoregressive models, which rely on many-step sampling procedures at inference to avoid mode blurring and often complicated masking/noising schedules (Section 2.1), end-to-end XMs generate with a single pass from the model and training XMs in the simplest case is just a short for loop with 3-5 lines of code (Algorithms 1 and 2).

Exploration as Building an Associative Memory. A generative model turns an input, usually noise drawn from a multivariate Gaussian, into a sample. Explorative Modeling searches over this noise to find which inputs the model should tie to which samples. Once trained, each region of noise acts as a key that retrieves one mode of the data as its value, so XMs behave much like associative memories. A larger K splits the noise into finer regions, so each mode gets its own instead of blurring together (Figure 2). This holds for every generative model, not just XMs. However, it *does not mean* that generative models simply memorize the training data. Large datasets act as a form of regularization: with far more examples than the model can store individually, it is forced to reuse parameters across them and learn the structure they share instead of the datapoints themselves, which results in generalization.

C Approach Details

Batching Forward XM. Algorithm 1 is written as a for loop for clarity, but in practice the K explored generations can be folded into the batch dimension and computed as a single larger forward pass. Because accelerators process larger batches efficiently, this parallelizes exploration and speeds up training considerably compared to looping over candidates one at a time. We provide batched code for Forward XM in the released code.

Other Ways to Explore. Drawing K fresh candidates per step is not the only way to do Forward XM. For example, generations could be cached by their class or conditioning and reused as candidates for later samples with the same condition—on ImageNet, caching recently generated samples for

each class would provide candidates essentially for free—though this works worse for continuous conditioning, where datapoints rarely share the exact same condition. We use fresh draws throughout, as this is the simplest approach to implement, is fairest when comparing across domains, and is less prone to collapse.

Gradients and Memory. Only the best candidate receives gradients (Equation 1), which can be implemented in two ways. In the *memory-saving* mode, all K candidates are forwarded without gradients, and only the best is re-forwarded with gradients to train on, keeping activation memory the same as standard training at the cost of one extra forward pass. In the *FLOP-efficient* mode, all K candidates are forwarded with gradients and only the lowest loss is backpropagated, avoiding the extra forward pass at the cost of storing activations for all K candidates. Switching between the two lets exploration adapt to whichever of FLOPs or memory is the bottleneck.

Exploration FLOP Cost. For transformers, a training step costs roughly $6ND$ FLOPs for N parameters and D tokens, where the forward pass costs $2ND$ and the backward pass $4ND$ [84]. Since only the best candidate is trained on, each additional explored candidate in Forward XM adds only a forward pass, so each additional mode explored costs roughly $\frac{1}{3}$ of a standard training step (XM- K costs $\frac{K+2}{3}$ standard steps in the FLOP-efficient mode, plus one more forward pass in the memory-saving mode). Exploring more modes in Reverse XM is far cheaper, as additional data targets add no forward passes, only extra loss computations—essentially one matrix multiplication scaling with the data dimension—which is negligible compared to the network’s FLOPs.

Generating the K Candidates. For all continuous models in this work, both end-to-end XMs and the hybrid XMs built on Diffusion/Flow and Jumpy models (Section 4.1), each explored candidate uses a different input noise draw; for the hybrids, this means the same data sample, timestep, and condition (including condition dropping for guidance when the underlying recipe uses it), with only the noise varying. For XMDLMs, we instead learn K discrete latent embeddings: each candidate samples one of these embeddings at random for every masked position, and the best-of- K selection is over the resulting latent-conditioned predictions.

Implementing Reverse XM. Reverse XM requires each generation to have a set of valid data targets to search over. With discrete conditioning, this comes almost for free through the dataloader: loading K datapoints per condition lets each generation pick its best match within the batch, at no extra generation cost. The same can be done with continuous conditioning, just involving additional tricks to load similar latents together. At larger scale, the whole dataset can instead be indexed in a vector database, letting each generation search all training data in roughly logarithmic time, so the number of modes explored can in principle reach the dataset size (we take this approach for the Reverse XM language modeling experiments described in Section A; see also Section 6).

D Experimental Details

A lot of the experimentation paragraph and takeaway style is inspired by [85]. Some of the figure designs in this paper follow those of Mode Forcing [19].

Model Sizes. All image and video generation models are transformers following the standard DiT size conventions [56], summarized in Table D.1.

Table D.1: **Model Sizes for Image and Video Generation.** Sizes follow the DiT conventions [56].

Size	Layers	Hidden Dim	Heads
Small	12	384	6
Base	12	768	12
Large	24	1024	16
XLarge	28	1152	16

Image Generation. All image generation experiments train class-conditional models on ImageNet 256×256 . Aside from the RAE experiments, we follow the SiT setup [29], with the exception of not using a horizontal flip augmentation. Images are encoded by the VAE [55] into $4 \times 32 \times 32$ latents, which with a patch size of 2 gives a 16×16 grid of patches, or 256 tokens per image (this is the token count behind the data scaling in Figure 5a). All models train with a batch size of 256 and a learning rate of $1e-4$, using 10k steps of linear warmup followed by cosine decay to 1M steps (or to 3M steps for the longer runs in Figure 5b), along with gradient clipping of 1.0 and weight decay of 0.01. For Diffusion models we use the Flow Matching formulation, and following the RAE and JiT papers [45, 86] we sample with 50 Heun steps [87] by default, which we found was enough to converge. For the FLOP scaling curves in Figure 5b we use Jumpy models [30], as they were more stable for longer runs and more performant. These experiments all report validation-set generative metrics, as they better measure overfitting and generalization.

RAE Image Generation. For the RAE experiments (Table 1 and Figure 4) we follow the RAE setup [45], building directly on their codebase and only adding exploration, including their batch size of 1024. We rely primarily on FDr⁶ [43] for these experiments, as FID at this level of performance is overfit and no longer tracks true sample quality [43, 44]. For guided results, both the RAE baseline and XRAE use AutoGuidance [26]: the RAE baseline uses a guidance scale of 1.42 with the released S model at epoch 14 as the guiding model; for XRAE-2, the best FDr⁶ uses a scale of 1.5 with the same released S model at epoch 14, while the best gFID uses a scale of 1.35 with an XM-2 L model at epoch 9.

Video Generation. Video generation experiments use the Something-Something V2 dataset [49] at 128×128 resolution, as higher resolutions required too much compute. We always model 10 frames, passing in frames 0, 1, and 9 as conditioning (simulating goal-conditioned world modeling [18]), and use a 3D video transformer [88] with Base model size and a patch size of 4, also chosen due to limited compute, as these experiments aim for fair comparisons rather than state-of-the-art results. Models train with a batch size of 256, a learning rate of $1e-4$, and weight decay of $1e-2$.

Individual Results in More Detail. Figure 7a uses Small models, while Figure 7b uses Base models. Figure 5 uses Base models for class-conditional image generation, where the baseline is the optimally tuned SiT recipe [29]. Figure 9 uses 4-step Jumpy models, plotting FVD over the course of training rather than the best performance reached. Figure 10a measures image generation improvements across Jumpy model sizes from Small through XLarge, and Figure 10b does the same for XLarge models across training steps. Figure 11 uses Base-sized image generation models.

Behavior Cloning. All Explorative Policy runs use XM-10. All Behavior Cloning models are CNNs rather than transformers, and we follow most of the Diffusion Policy setup [17]. However, we use a newer version of robomimic [89], so results are not perfectly comparable to those originally reported; we therefore reproduce Diffusion Policy’s results under our setup (Table 2).

Goal-Conditioned World Modeling. All Explorative World Model runs use XM-10. We evaluate on the single-task Maze2D setting from Table 1 of the Diffuser paper [18], training for 1M steps instead of 2M and using a goal radius of 0.45. As with Behavior Cloning, for a fair setup, we reproduce Diffuser’s results under our setup (Table 3). Our end-to-end explorative world models’ NFE exceeds one only with a recurrent block [65]: U-Maze recurs over the full depth 3 times (NFE 4), Large over the middle 3 times (NFE 1.9), Medium not at all.

Language Modeling. Our MDLM [27] experiments use a context length of 256 and pretrain for 210k steps with a batch size of 64. Models are xxs-sized: 6 transformer blocks, 6 attention heads, and an embedding dimension of 384. We report generative perplexity using the Qwen2.5-1.5B base model (not post-trained) [90] as the evaluator, as it is a much better oracle than the commonly used GPT-2 [91], though GPT-2 results confirmed the same trends. Results are primarily across an entropy range of 5.0 to 5.7 and the sampling step counts shown in Figure 8. We also confirmed that exploration’s gains hold at larger scale as well as for infilling tasks.

E Related Works

E.1 End-to-End Generative Modeling

We call a generative model end-to-end when sampling during training, if done at all, is the same as sampling during inference (Section 2.2). Contrastive generative models—including GANs [20] and contrastive-divergence EBMs [21, 22]—have long been end-to-end, but have struggled with scaling as well as reconstructive models [92], so we focus on reconstructive generative models as the primary target for end-to-end generation. Among reconstructive models, normalizing flows [93] are commonly seen as end-to-end, as they train an invertible map by exact likelihood and sample by inverting that map in a single pass. However, they fall short of our definition of end-to-end generative models (Section 2.2), as training only ever evaluates the data-to-noise direction, so the noise-to-data sampling pass used at inference is never simulated during training. Additionally, normalizing flows have struggled to scale as standalone generators: a continuous bijection cannot split its unimodal base into well-separated modes without leaving thin bridges of density between them [94]. The flows that generate well today escape this by importing factorization, often stacking autoregressive blocks [95, 96], which only widens this train-inference gap, as training never simulates the sequential recursion used at inference. In contrast, recent reconstructive approaches have progressively narrowed the train/inference gap: consistency models [14] learn few-step samplers by enforcing self-consistency along the probability-flow ODE trajectory (via distillation or from scratch), and MeanFlow [15] regresses the flow marginal mean to enable single-step generation. These methods still maintain a mismatch, however, since training primarily samples portions of the trajectory rather than the full generative process used at inference. Mode Forcing explains why this gap has been hard to close: factoring generation into near-unimodal steps is exactly what lets reconstruction losses avoid mode blurring, so existing scalable reconstructive models that factor generation are fundamentally unable to be end-to-end. XMs enable resolving this challenge by factoring the training loop.

E.2 Coupling

A natural attempt to avoid mode blurring without factorizing generation is to choose a smarter coupling. Optimal Transport [39, 82] reduces path crossings by minimizing geometric distance, and minibatch OT [97] approximates this per batch—but the resulting coupling is a biased proxy for the global one, and its gains narrow at scale [98]. In our own experiments they reverse entirely (Section A), as trying to use OT couplings actually results in worse performance, which we attribute to minibatch bias and OT’s exogenous, model-agnostic assignment. Model-Aligned Coupling (MAC) [99] selects training pairs by model prediction error in addition to geometry, improving few-step flow matching, but is specific to trajectory regularization and not designed to raise generative expressivity or handle multimodal distributions more broadly.

E.3 Explorative Modeling Based Methods

We do not claim to invent best-of- K : generating several candidates and keeping the best is a simple idea that has appeared across many works, from winner-take-all training objectives [33–35, 100, 101] to inference-time best-of- N selection, where candidate generations are reranked using a learned reward model [102], a trained verifier [103], or the model’s own energy [69]. Rather, what we claim is the realization of why this idea matters for generative modeling: exploration is a fundamentally different way of handling multimodal distributions from the generation factoring of modern reconstructive generative models (described in Section 2), working as a scalable latent variable search that keeps the loss minimizer on individual modes rather than their blurred average (Section 3.1). In particular, exploration offers a general way to increase generative expressivity, which existing models otherwise fix at design time through how they factor generation. It is this understanding that lets us use exploration deliberately, as a new scaling axis that raises the generative expressivity of existing generative models, and as a standalone approach for end-to-end generation through Forward and Reverse XM. Through this lens, several existing methods can be seen as realizing a form of exploration, which we describe next.

Multiple Choice Learning [33, 100] trains an ensemble of K predictors under an oracle loss, gating gradients through whichever member has the lowest error on each example, an idea later extended to single networks with multiple prediction heads [101]. The goal is ensemble diversity for downstream

oracle selection, not generative modeling, though it shares the structural idea of backpropagating only through the minimum-loss prediction.

Implicit Maximum Likelihood Estimation (IMLE) [34] draws a pool of model samples, matches each data point to its nearest sample via fast approximate nearest-neighbor search, and trains on those pairs, with theoretical guarantees that this recovers MLE under mild conditions. Structurally, XMs generalize IMLE: IMLE is a specific instance of end-to-end Forward XM with a shared global sample pool in place of per-step candidates, and both minimize the expected distance from each data point to its nearest model output. Self-Organizing Generative Models (SOG) [35] realize the same objective conditionally—sampling several latent codes per datapoint and training only through the lowest-loss one, another instance of end-to-end Forward XM—interpreting this procedure as encoder-free hard-assignment maximum likelihood. IMLE attributes its success to performing *implicit* maximum likelihood—the argument that nearest-sample matching is a likelihood-free proxy for log-likelihood maximization. But we argue this perspective misidentifies the working mechanism—if maximum likelihood were what makes IMLE work, then Reverse XM, which fixes one model output and trains on its nearest data point rather than the reverse, optimizing the mode-seeking reverse KL rather than any likelihood (Section F.2), would have no reason to capture modes—yet it does. What the two methods share instead is exploration: drawing multiple candidates so the reconstruction loss minimizer lands on a mode rather than a blurred average. This coincides with the main hypothesis behind Mode Forcing [19], that modern reconstructive generative models are not fundamentally about MLE or similar objectives but rather about designing a reconstructive objective where the loss minimizer does not blur modes. Explorative Modeling makes this principle general—it applies to any generative model and loss, not just end-to-end implicit models with Euclidean distance, covers both forward and reverse directions, and can be layered onto existing generative models as a scaling axis.

F Additional Theory

F.1 Generative Expressivity Details

Here we expand on the definition of generative expressivity E from Section 2.1, which follows Mode Forcing [19]. The mode count $M(q)$ is the number of strict local maxima of a distribution q , or for discrete distributions the cardinality of its support, and $P_\theta(\cdot | c)$ is the distribution induced by the model’s inference-time sampling procedure. Generative expressivity is

$$E \triangleq \sup_{p^*, c} \sup_{\theta^* \in \arg \min_\theta \mathcal{L}(\theta)} M(P_{\theta^*}(\cdot | c)),$$

where the outer supremum over data distributions makes E a property of the training objective itself rather than of any particular dataset, the inner supremum reads E as a capacity—the ceiling the objective permits, analogous to parameter count—and minimizers range over all densities (we assume the nonparametric optimum is realizable). Under direct Bregman regression the minimizer is unique for every p^* (the conditional mean, a point mass), so $E = 1$ regardless of parameter count. Under smooth Forward XM with K candidates, $E \geq K$: taking p^* with K modes separated as in Proposition 3, every minimizer covers each mode with its own region of mass and so retains at least K modes, one maximum per mode (Proposition 3); the separation condition is what makes the mode neighborhoods resolvable. Achievement is a per-distribution statement: on a p^* with $M^*(c) \leq K$ such modes, minimizers retain all $M^*(c)$ of them, so unimodal data yields unimodal minimizers at any K . Finally, these claims belong to the smooth objective—the hard min differs by at most $\log K$ at finite K , but its large- K limit is a coverage objective without distributional control (Section F.2), so there E reports unbounded capacity without implying sample quality. We otherwise inherit the regularity assumptions of Mode Forcing [19]—densities exist, reconstruction losses are Bregman divergences, and M counts modes irrespective of their mass or separation—and refer to [19] for the mean-blurring lemma behind $E = 1$.

Scope of E . E is an at-optimum capacity: for direct regression it is pinned exactly ($E = 1$), and for smooth Forward XM Proposition 3 delivers a lower bound with per-distribution achievement. Applied to *consistent* factored objectives, however, this at-optimum reading saturates. Autoregression under cross-entropy and continuous-time diffusion sampled exactly admit nonparametric minimizers whose sampling distribution is p^* itself (under standard regularity), so at their optima they inherit every mode of the data and E reaches the most the sample space allows— V^L for autoregression

over L tokens with vocabulary size V , unbounded for continuous diffusion (read at the loss scale this count is finite [19], and what a trained model realizes in practice is far smaller, small enough to bind). Nor do finite sampling steps restore a meaningful cap: even a two-step sampler’s optimum can retain arbitrarily many well-separated modes, albeit with no control over their mass—whereas Proposition 3 at least guarantees every mode is covered. The at-optimum definition therefore registers no deficit for any consistent objective, because at the optimum there is none—it separates direct regression from factored models, but not factored models from one another.

Per-Prediction Expressivity. The deficit factored models do carry lives at the level of each prediction. A factored model is trained as many small reconstruction problems, and each carries its own ceiling—a *per-prediction expressivity*, how much of its own target’s structure a single step’s minimizer can retain. For an MSE step this ceiling is one mode, the conditional mean of its target; a parallel discrete decoder retains full per-position conditionals but predicts them independently, so no single step retains dependence among the tokens it reveals. Factoring does not raise these per-prediction ceilings; what it changes is each prediction’s *residual multimodality*—how many valid targets compete for it during training—which richer conditioning and finer factoring shrink but never quite remove (Section 2.1). Mode Forcing makes this picture quantitative, defining each prediction’s ceiling e_t and its residual multimodality $M_t(c_t)$ pointwise in the conditioning, so a prediction pays $\max(M_t(c_t) - e_t, 0)$ wherever it is asked; these shortfalls are not claimed to sum to a total [19]. This residue costs a model in one of two ways. Where sampling exposes the compromise directly it surfaces in generations, as blur or incoherence: direct regression, few jumps, coarse steps, or many positions unmasked at once. Where sampling does not, the same compromise is paid during training instead: competing targets pull each prediction against itself, whether the loss blurs them or must spread over them, a cost set by the training objective and untouched by the sampler. Relieving this training-time cost, we hypothesize, is why exploration can still help models whose sampling steps have already converged (Figures 7 and 8) and why XMs converge faster (Figure 4). So when we say existing models fix generative expressivity at design time, we mean the per-prediction notion: for a given data distribution the factorization sets the residue each prediction faces, the objective sets what each retains of it, and neither changes with parameters or data. Exploration attacks the residue directly, either by letting one input carry K distinct predictions through a latent (end-to-end XMs and our XMDLMs), or by searching which noise draw a datapoint trains under so that fewer competing targets land on the same prediction (our continuous hybrids; Section C). In the budget’s terms, the first route raises e_t —to at least K for a squared-error prediction by Proposition 3, and heuristically for richer heads, where the K latent-conditioned candidates turn the step’s product head into a mixture that can carry dependence among the positions it reveals—while the second shrinks the residue M_t each prediction effectively faces. It should therefore help most where the residue is largest—direct regression, few-jump Jumpy models (the jumps-for-exploration substitution of Figure 11), and the sparse, any-order conditioning of masked language models. It should help least for autoregression over discrete tokens, whose full-prefix conditioning leaves each next-token target nearly unimodal and whose per-token cross-entropy retains a full conditional rather than a mean, consistent with autoregressive language models proving harder to improve (Section 7). Continuous autoregression, where a plain per-step regression would fit each next element by the mean of its valid values, should benefit like the other continuous families. Even a small residue does not make exploration worthless, however: our video generation results improved significantly despite weak multimodality, which we hypothesize reflects surplus generative expressivity aiding optimization and generalization much as overparametrization does (Section 5).

F.2 What Forward and Reverse XM Optimize

Overview. This subsection makes the claims of Section 3.2 precise, and involves two distributions: the data distribution p^* and the distribution of the model’s generations g_θ . Because squared error is, up to a constant, the negative log of a Gaussian of width σ , the loss effectively blurs whichever distribution it is applied to, so we write p_θ for the model’s generations blurred by this Gaussian and p_σ^* for the data blurred in the same way. In their smooth forms, Forward and Reverse XM minimize

$$\text{KL}(p^* \parallel p_\theta) + H(p^*) \quad \text{and} \quad \text{KL}(g_\theta \parallel p_\sigma^*) + H(g_\theta),$$

respectively. These identities hold at every K with the blurred densities replaced by their K -sample versions (Propositions 1 and 2), and hold exactly as written in the large- K limit. The two objectives differ only in the entropy each carries. For Forward XM, this entropy is the *data*’s, a constant the

model cannot change, so Forward XM performs maximum likelihood—of its K -candidate mixture—at every K . For Reverse XM, this entropy is instead the *model's* own, which the model can lower by shrinking its spread, so Reverse XM drifts toward collapse by itself and needs an entropy bonus to become the pure reverse KL. The remainder of this subsection states and proves these claims.

Setup. We ignore additive constants throughout, as they change neither gradients nor minimizers. We also suppress any conditioning (such as a class label or timestep), as every statement below holds per condition. The loss between a generation $\hat{y}$ and a datapoint x is then $-\log k_\sigma(\hat{y}, x)$, where for squared error the kernel is a Gaussian centered on the generation, $k_\sigma(\hat{y}, x) = \mathcal{N}(x; \hat{y}, \sigma^2 I)$. Blurring each side by this kernel gives the two densities of the overview,

$$p_\theta = g_\theta * k_\sigma, \quad p_\sigma^* = p^* * k_\sigma,$$

so that p_θ represents the model as the loss sees it, and p_σ^* represents the data in the same way. Finally, we analyze the smooth form of exploration, which scores the K explored candidates by their average kernel, $-\log \frac{1}{K} \sum_i k_\sigma(\hat{y}_i, x)$, rather than by the best one alone. This soft min differs from the hard min by at most $\log K$, and we describe where the two diverge later in this subsection.

Proposition 1 (Forward XM is maximum likelihood). *At every K , the smooth Forward XM objective (the soft counterpart of Equation 1)*

$$L_F(\theta) = \mathbb{E}_{x \sim p^*} \mathbb{E}_{\hat{y}_{1:K} \sim g_\theta} \left[-\log \frac{1}{K} \sum_i k_\sigma(\hat{y}_i, x) \right]$$

is exactly the expected negative log-likelihood of the data under the mixture of the model's own K explored generations, $\frac{1}{K} \sum_i k_\sigma(\hat{y}_i, \cdot)$, so Forward XM performs maximum likelihood at every K . As $K \rightarrow \infty$ this mixture converges to the blurred model p_θ , so minimizing L_F becomes minimizing $\text{KL}(p^ \| p_\theta)$, recovering p^* as $\sigma \rightarrow 0$.*

Proof sketch. For a fixed draw of candidates, $\frac{1}{K} \sum_i k_\sigma(\hat{y}_i, \cdot)$ is a normalized density, so the inner term is exactly the negative log-likelihood of x under this mixture, and subtracting the constant data entropy $H(p^*)$ leaves the forward KL to the mixture. The same average is an unbiased estimate of $p_\theta(x)$, so by Jensen L_F upper-bounds $\mathbb{E}_{p^*}[-\log p_\theta]$ for every K , tightening as K grows by the argument of IWAE [104] and converging by the law of large numbers (with domination), where $\mathbb{E}_{p^*}[-\log p_\theta] = \text{KL}(p^* \| p_\theta) + H(p^*)$. $\square$

Remark (the optimum at fixed σ). Over all generator densities, the optimum is the KL projection of p^* onto the blurred family $\{g * k_\sigma\}$; it equals p^* exactly when the σ -deconvolution of p^* exists as a density, and in general the match becomes exact only as $\sigma \rightarrow 0$.

The role of K . Since Forward XM performs maximum likelihood at every K , what K controls is the density each draw fits. At $K=1$ this density is a single Gaussian of width σ , whose best fit (for squared error) is a point mass at the data mean, the blurred mean. At larger K it is a K -component mixture that can place mass on many modes, becoming the full blurred model p_θ as $K \rightarrow \infty$. At inference, however, the model draws a single sample from g_θ , which the loss scores only through its blur p_θ ; from that fixed density's view, L_F is an upper bound on its negative log-likelihood that tightens monotonically as K grows [104]. The gap between the two views is a Jensen gap: because the mixture's components are sampled from g_θ rather than placed freely, finite K penalizes draws that miss a mode. When per-draw scores concentrate this gap is $O(1/K)$ and merely favors slightly lower-variance generators; with well-separated modes and few candidates, misses are costly enough that minimizers hedge mass between modes (Proposition 3), an effect that fades as K outgrows the mode count. Exploration therefore turns a unimodal regressor into a multimodal likelihood model.

The hard min. The hard min actually implemented differs from the soft one by at most $\log K$, and the difference matters most in the limit. As $K \rightarrow \infty$, the hard objective only asks that every datapoint have some generation arbitrarily close to it, so any model whose generations cover the data's support becomes optimal, regardless of how it spreads mass over that support. The hard min is therefore a *coverage* objective, which shares the mode-covering character of the forward KL without pinning down the density, and the clean likelihood statements are those of the smooth form above.

Proposition 2 (Reverse XM targets the reverse KL, but collapses without an entropy term). *At every K , the smooth Reverse XM objective (the soft counterpart of Equation 2)*

$$L_R(\theta) = \mathbb{E}_{\hat{y} \sim g_\theta} \mathbb{E}_{x_{1:K} \sim p^*} \left[-\log \frac{1}{K} \sum_i k_\sigma(\hat{y}, x_i) \right]$$

equals, in expectation over the sampled targets, $\text{KL}(g_\theta \parallel \hat{p}_\sigma) + H(g_\theta)$, where $\hat{p}_\sigma = \frac{1}{K} \sum_i k_\sigma(\cdot, x_i)$ is the kernel density estimate of the K data targets, converging to $\text{KL}(g_\theta \parallel p_\sigma^) + H(g_\theta)$ as $K \rightarrow \infty$. Because each generation is scored on its own, independently of how often the model produces it, L_R is linear in g_θ , so its infimum is approached by collapse onto a point mass (at $K=1$, the data mean) and bare Reverse XM does not recover p^* . Adding an entropy bonus, i.e. minimizing $L_R - H(g_\theta)$, cancels the entropy term and leaves the pure reverse KL, minimized as $K \rightarrow \infty$ at $g_\theta = p_\sigma^*$, reaching p^* as $\sigma \rightarrow 0$.*

Proof sketch. For a fixed draw of targets, $\hat{p}_\sigma$ is a normalized density, so the inner expectation is the cross-entropy between g_θ and $\hat{p}_\sigma$, which decomposes as $\text{KL}(g_\theta \parallel \hat{p}_\sigma) + H(g_\theta)$; by the law of large numbers $\hat{p}_\sigma(\hat{y}) \rightarrow p_\sigma^*(\hat{y})$, giving the limit. The objective weights each generation by a score that does not depend on how often the model produces it, so it is linear in g_θ , and its infimum over densities is approached by densities concentrating toward a point mass at the minimizer of the per-generation cost (the data mean at $K=1$, a global mode of p_σ^* as K grows). Subtracting $H(g_\theta)$ in the limit leaves the reverse KL, minimized at $g_\theta = p_\sigma^*$. $\square$

Where the blur sits. One further asymmetry is where the blur sits. Forward XM compares the raw data against the blurred model, whereas Reverse XM compares the raw model against the blurred data, and as $\sigma \rightarrow 0$ both become comparisons between the raw model and the raw data, in opposite directions.

On the ELBO. Only the Forward side carries a genuine evidence bound, as the inner $\log \frac{1}{K} \sum_i k_\sigma(\hat{y}_i, x)$ is, in expectation, an importance-weighted lower bound on the model log-likelihood $\log p_\theta(x)$, as in IWAE [104]. The entropy-corrected Reverse objective $L_R - H(g_\theta)$ is instead a variational free energy for the target p_σ^* as $K \rightarrow \infty$, while bare L_R keeps only its energy term, which is why it collapses and bounds no model likelihood. For this reason, we describe Reverse XM as reverse-KL or variational rather than as an ELBO.

Assumptions. These statements require (i) the loss to be, up to an additive constant, the negative log of a kernel whose normalizer is independent of $\hat{y}$ and θ (squared error $\leftrightarrow$ Gaussian; for non-normalizable losses such as bounded or perceptual ones the reading fails and these statements do not apply), (ii) the smooth surrogate rather than the hard min, (iii) $K \rightarrow \infty$ only to replace the K -sample mixtures with the blurred densities p_θ and p_σ^* , and $\sigma \rightarrow 0$ to reach p^* exactly, and (iv) mild regularity: a bounded, symmetric, translation-invariant kernel (so p_σ^* is a density), finite expected loss and differential entropies, domination to justify the limit interchanges, a continuous loss vanishing only at $\hat{y} = x$ (for the hard-min limit), and minimizers ranging over all densities. Finally, these statements analyze end-to-end XMs, but because conditioning is suppressed throughout, they extend to any single prediction of a factored model whose K explored candidates share their conditioning and target, with p^* read as that prediction’s target conditional. For discrete predictions the kernel is the candidate’s own softmax, so Proposition 1’s mixture-likelihood reading carries over with no blur (σ plays no role), which covers our XMDLMs; the separation-based Proposition 3 and the kernel-density reading of Proposition 2 are squared-error statements and do not transfer. The noise-searching form of our continuous hybrids instead pairs each candidate with its own corruption of the datapoint, a coupling search rather than a best-of- K against a fixed target, and we leave its analysis, along with how per-prediction gains compose across a sampler’s steps, to future work.

Proposition 3 (Expressivity of smooth Forward XM under separation). *Suppose p^* is a mixture of $M^* \leq K$ unimodal components with near-equal masses, each concentrated at scale σ or below, whose modes are pairwise separated by distances $\Delta \gg \sigma$, with $\Delta^2/\sigma^2 \gg (\log K)/\min_m w_m$ for component masses w_m , and let the assumptions of this section hold. Then every minimizer g_θ of the smooth Forward XM objective L_F places positive mass in a neighborhood of each mode of p^* —so $M(g_\theta) \geq M^*$, and taking $M^* = K$ gives $E \geq K$ —though at finite K minimizers may also hedge mass between modes, as insurance against candidate draws that miss a mode. Once candidates sufficiently outnumber modes, $K(1 - \min_m w_m)^K \Delta^2/\sigma^2 \lesssim 1$, misses are rare enough that hedging*

*no longer pays: all but a negligible fraction of mass sits at the modes, and the blurred density $p_\theta = g_\theta * k_\sigma$ has exactly M^* strict local maxima, one per component—equivalently, what the model deploys, g_θ , carries exactly M^* modes read at the loss scale.*

Proof sketch. By Proposition 1, minimizing L_F is maximum likelihood of p^* under the mixture of K candidates drawn i.i.d. from g_θ . If g_θ assigns a mode zero mass, no draw ever covers it, and the objective then pays of order $w_m \Delta^2 / \sigma^2$ for a mode of mass w_m , while concentrating that mass elsewhere gains at most $\log K$ in likelihood, plus what it saves by making misses rarer at the remaining modes—with near-equal masses a constant fraction of what the dropped mode costs—so neither term covers the loss, and every minimizer covers every mode with positive mass. The same $\log K$ budget also forces concentration: spreading mode m ’s share to width ρ costs its covered datapoints of order $w_m \rho^2 / \sigma^2$ against a total possible gain of $\log K$, so under the separation condition each mode’s mass sits within $o(\Delta)$ of its center. Each region’s peak density therefore dominates the between-mode bridges, whose height is capped by the same budget, and hedge mass—lying at distance $\sim \Delta$ from the peaks at lower density—leaves each peak a strict local maximum, hence $M(g_\theta) \geq M^*$; concentration to $O(\sigma)$ and the exact count M^* require the rare-miss condition below. Because candidates are sampled rather than placed, a draw misses a mode of weight w with probability $(1 - w)^K$, at a cost of order Δ^2 / σ^2 , so minimizers also keep insurance mass *between* modes—the bridges visible at small K in Figure 2—which likewise perturbs the optimal mode weights. Enumerating K learned discrete latents instead puts a candidate at every mode on every step, so misses never occur and the bridges vanish at any K , though K latents alone then cap how many distinct outputs a condition can produce (it’s possible a mixture of discrete and continuous is optimal). Under the additional condition, misses are rare enough that insurance costs more in diluted likelihood than it saves, so all but a negligible fraction of mass concentrates in the mode neighborhoods (the concentration hypothesis leaves exponentially little of p^* elsewhere). Convolving with k_σ then merges each neighborhood’s sub- σ structure into a single bump, with exponentially small cross-terms and no spurious maxima between components, so p_θ has exactly M^* strict local maxima. $\square$

G Frequently Asked Questions

Here we answer some common questions about Explorative Modeling.

What’s the main takeaway? How is this paper ‘novel’ if it’s just best-of- K ? Introducing best-of- K is not the central contribution or claim of this paper, as sampling candidates and keeping the best has appeared many times before [33–35] (Section E). What is new is the realization of what this simple loop does: it increases generative expressivity without factoring generation, which is a completely different factorization axis from popular generative models such as Autoregression and Diffusion (Figure 1). The paper’s central messages follow from this realization: generative expressivity is worth scaling through exploration alongside model parameters and data as a new pretraining axis, as it improves both the performance and generalization of existing generative models (a way of trading training compute for generalization). Because generative expressivity increasingly becomes the bottleneck as parameters and data grow, gains from exploration **grow with scale** rather than saturate, making exploration more important as scale increases. Exploration can also stand in for factoring generation, turning how end-to-end a model is into something we can scale rather than a fixed design choice, all the way to fully end-to-end XMs that match diffusion baselines at a fraction of the inference compute. Finally, best-of- K is just one implementation of exploration—we have already seen other implementations succeed (Reverse XM, Section A), and expect more versions to eventually become viable (e.g., gradient-based search, Section 6).

Does exploration make inference more expensive? No. Exploration happens entirely during training, so inference is unchanged. The added cost shows up as more expensive training instead (for Forward XM, for Reverse XM the added training cost is often negligible), and as shown in Figure 4b, this cost is well worth it.

Is exploration really a new “axis” if it just costs more compute? Scaling parameters or data costs compute too—no scaling axis is free. The question is whether exploration is a *good* way to spend compute, which is the same question compute-optimal scaling [16] asks when dividing a budget between parameters and data. Our FLOP-matched comparisons answer it directly: models

that explore are significantly more compute efficient than models that just train for longer (Figures 4b and 5b), and the FLOP-optimal amount of exploration grows with scale, just as the compute-optimal parameter count does.

Why the name Explorative Modeling? We believe the idea of exploration captures the intuition of trying to understand the model’s loss landscape with respect to the different modes. However, exploration is a type of search, and we considered naming the approach after search directly. We decided against this because gradient descent is already a search over parameters, and because search is now strongly associated with inference, where “scaling search” would sound like inference-time thinking/reasoning rather than a training axis. Every candidate term is overloaded in some way; we found exploration to be the best general term, covering Forward XM, Reverse XM, and the gradient-based variants discussed in Section 6.

When does exploration help? Exploration helps whenever generation is conditioned on a latent variable the model can search over, such as the input noise in a diffusion model. We have found exploration helps most when generative expressivity is a bottleneck, although this does not have to be the case—our video models improved significantly even though the generated distribution was not very multimodal (Section 5).

Does training on the model’s own generations cause collapse? No, because the training targets are always real data: the model’s generations only decide which datapoint each noise input gets paired with. In Forward XM, every datapoint still receives a training signal, so no part of the data distribution can be dropped; in fact, Forward XM is maximum likelihood (Section F.2). Collapse is a genuine concern for Reverse XM, which is mode-seeking, and is exactly why it needs an entropy term. A simple coverage constraint served as this entropy term when training our Reverse XM language models (Section A).

How should the amount of exploration K be chosen? This deserves more study, just like studying how parameters and data should be scaled together has been important [16]. The right amount of exploration is problem dependent, as some distributions have many more modes to capture and so rely on exploration more than others. In our experiments, higher K values became better with scale, both within a single run, where the FLOP-optimal amount of exploration grows over the course of training (Figures 4b and 5b), and across runs, where gains from exploration grow with model and data scale (Figure 10). Exploration is also flexible: models with more training compute available can increase exploration to buy generalization and more end-to-end generation. Given these many tradeoffs, we recommend sweeping K across 1 (the baseline), 2, 3, and 5 to start, sweeping 8 and 12 if compute permits, and pushing even further if gains continue. When compute is tight, we recommend starting with small values like $K = 2$ or 3, as they are cheap and improve performance significantly (Figure 4).

Is Explorative Modeling a form of reinforcement learning? No. Explorative Modeling is a generative modeling objective, unrelated to reinforcement learning. RL uses “exploration” for agents trying varied actions to discover reward [105], which differs in mechanism and purpose from XMs’ within-step candidate sampling, though both share the intuition that trying many options reveals information a single choice cannot.